

# Unit 2

## **Introduction to NLP**

Introduction to NLP, NLP Pipeline,

Lexical Analysis: Words and Their Components, Regular Expressions, Text Normalization,

Morphological Analysis: Morphological Models, Finite-State Automata, Morphological Parsing, Parts of Speech and Named Entities- Hidden Markov Models

Syntax Analysis: Parsing Natural Language, Treebanks: A Data-Driven Approach to Syntax, Models for Ambiguity Resolution in Parsing, Efficient parsing for context-free grammars (CFGs). Statistical parsing and probabilistic CFGs (PCFGs).

# Natural Language Processing

Natural Language Processing (NLP) refers to the branch of computer science—and more specifically, the branch of [artificial intelligence](#) concerned with giving computers the ability to understand text and spoken words in much the same way human beings can.

NLP combines computational linguistics—rule-based modeling of human language—with statistical, machine learning, and deep learning models. Together, these technologies enable computers to process human language in the form of text or voice data and to ‘understand’ its full meaning, complete with the speaker or writer’s intent and sentiment.

# Natural Language Processing (contd)

- NLP drives computer programs that translate text from one language to another, respond to spoken commands, and summarize large volumes of text rapidly—even in real time.
- One may have interacted with NLP in the form of
  - voice-operated GPS systems,
  - digital assistants,
  - speech-to-text dictation software,
  - customer service chatbots, and
  - other consumer conveniences.
- But NLP also plays a growing role in enterprise solutions that help streamline business operations, increase employee productivity, and simplify mission-critical business processes.

# **NLP tools and approaches**

# 1) Python and the Natural Language Toolkit (NLTK)

- The Python programming language provides a wide range of tools and libraries for attacking specific NLP tasks. Many of these are found in the Natural Language Toolkit, or NLTK, an open source collection of libraries, programs, and education resources for building NLP programs.
- The NLTK includes libraries for many of the NLP tasks listed above, plus libraries for subtasks, such as sentence parsing, word segmentation, stemming and lemmatization (methods of trimming words down to their roots), and tokenization (for breaking phrases, sentences, paragraphs and passages into tokens that help the computer better understand the text).
- It also includes libraries for implementing capabilities such as semantic reasoning, the ability to reach logical conclusions based on facts extracted from text.

## 2) Statistical NLP, machine learning, and deep learning

- The earliest NLP applications were hand-coded, rules-based systems that could perform certain NLP tasks, but couldn't easily scale to accommodate a seemingly endless stream of exceptions or the increasing volumes of text and voice data.
- Statistical NLP combines computer algorithms with machine learning and [deep learning](#) models to automatically extract, classify, and label elements of text and voice data and then assign a statistical likelihood to each possible meaning of those elements.
- Today, deep learning models and learning techniques based on convolutional neural networks (CNNs) and recurrent neural networks (RNNs) enable NLP systems that 'learn' as they work and extract ever more accurate meaning from huge volumes of raw, unstructured, and unlabeled text and voice data sets.

## Role of NLP in CS

Natural Language Processing is a form of AI that **gives machines the ability to not just read, but to understand and interpret human language.**

With NLP, machines can make sense of written or spoken text and perform tasks including speech recognition, sentiment analysis, and automatic text summarization

# Why is NLP important?

## 1. Large volumes of textual data

Natural language processing helps computers communicate with humans in their own language and scales other language-related tasks. For example, NLP makes it possible for computers to read text, hear speech, interpret it, measure sentiment and determine which parts are important.

Today's machines can analyze more language-based data than humans, without fatigue and in a consistent, unbiased way. Considering the staggering amount of unstructured data that's generated every day, from medical records to social media, automation will be critical to fully analyze text and speech data efficiently.

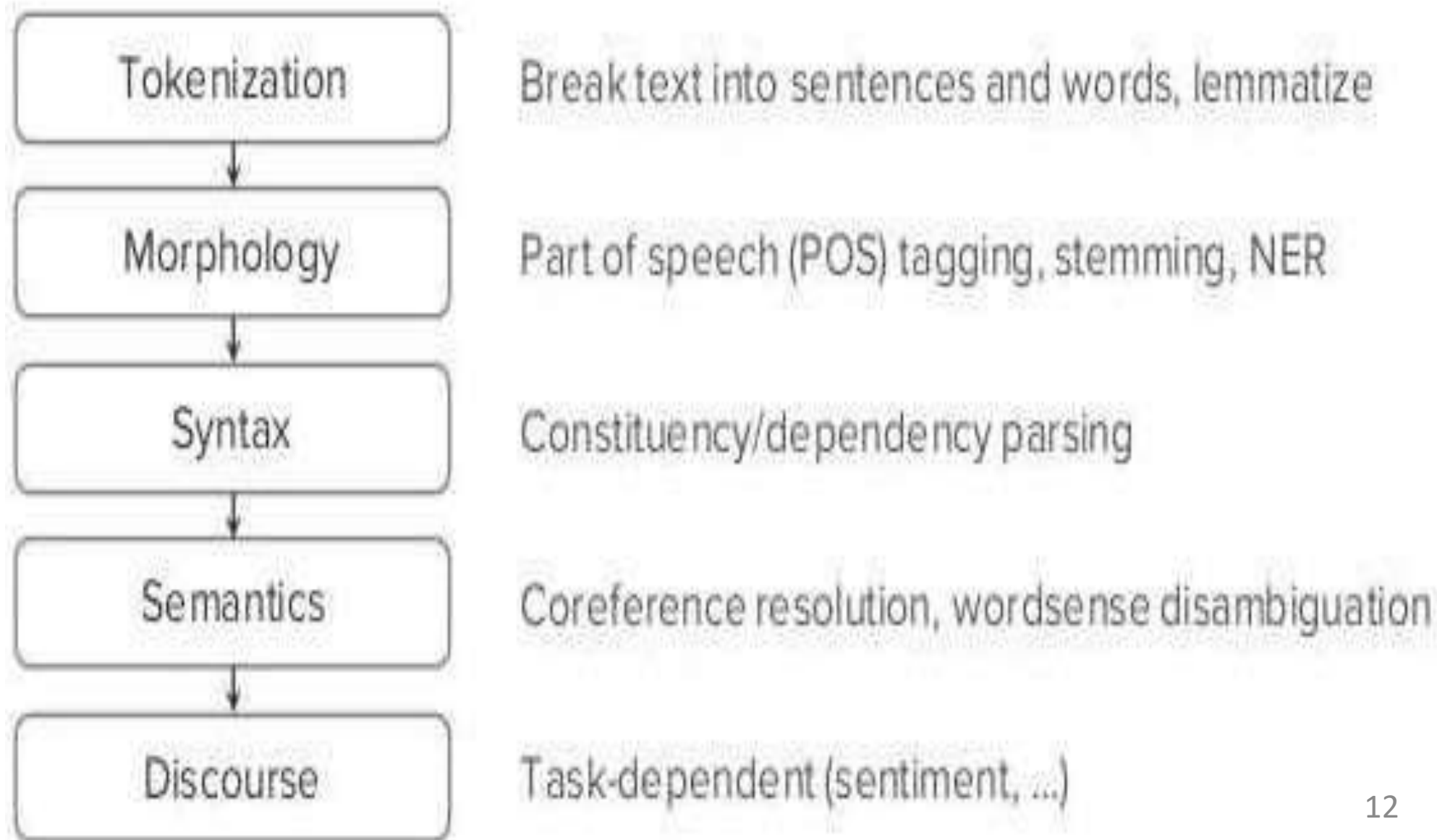


## 2. Structuring a highly unstructured data source

- Human language is astoundingly complex and diverse. We express ourselves in infinite ways, both verbally and in writing. Not only are there hundreds of languages and dialects, but within each language is a unique set of grammar and syntax rules, terms and slang. When we write, we often misspell or abbreviate words, or omit punctuation. When we speak, we have regional accents, and we mumble, stutter and borrow terms from other languages.
- While supervised and unsupervised learning, and specifically deep learning, are now widely used for modeling human language, there's also a need for syntactic and semantic understanding and domain expertise that are not necessarily present in these machine learning approaches.
- NLP is important because it helps resolve ambiguity in language and adds useful numeric structure to the data for many downstream applications, such as speech recognition or text analytics.

# NLP Pipeline

# “Classical” NLP Pipeline



# 1. TOKENIZATION

Tokenization is the first step in any NLP pipeline. It has an important effect on the rest of your pipeline.

**Tokenization** is the process of tokenizing or splitting a string, text into a list of tokens.

A tokenizer breaks unstructured data and natural language text into chunks of information that can be considered as discrete elements. The token occurrences in a document can be used directly as a vector representing that document.

This immediately turns an unstructured string (text document) into a numerical data structure suitable for machine learning. They can also be used directly by a computer to trigger useful actions and responses. Or they might be used in a machine learning pipeline as features that trigger more complex decisions or behavior.

## Lexical Analysis

- Lexical analysis separates tokens from the language statements based on “Pattern Matching Principle”.
- A token (string of characters) is the smallest unit with some logical meaning, which cannot be further decomposed.
- Regular expression is used to specify text strings in all sorts of text processing and information extraction applications.
- “Finite State Automata” is a basic mathematical foundation used extensively in Computational Linguistics.
- Variations / different forms of Finite state automata – state transducers, HMM, N grams

# Regular Expression (RE)

- RE is a Language for specifying text search strings
- RE is an algebraic notation for characterizing a set of strings/ sequence of symbols
- RE can be used to define a Language in a formal way

Eg. Variable name in a programming language

$L(L|D|S)^*$

a, abc, a10, min\_count, ....

here  $L \rightarrow \{a..zA..Z\}$

$D \rightarrow \{0..9\}$

$S \rightarrow \{\$|_\}$

Eg. A person's name in a Natural language

$L1L^*$

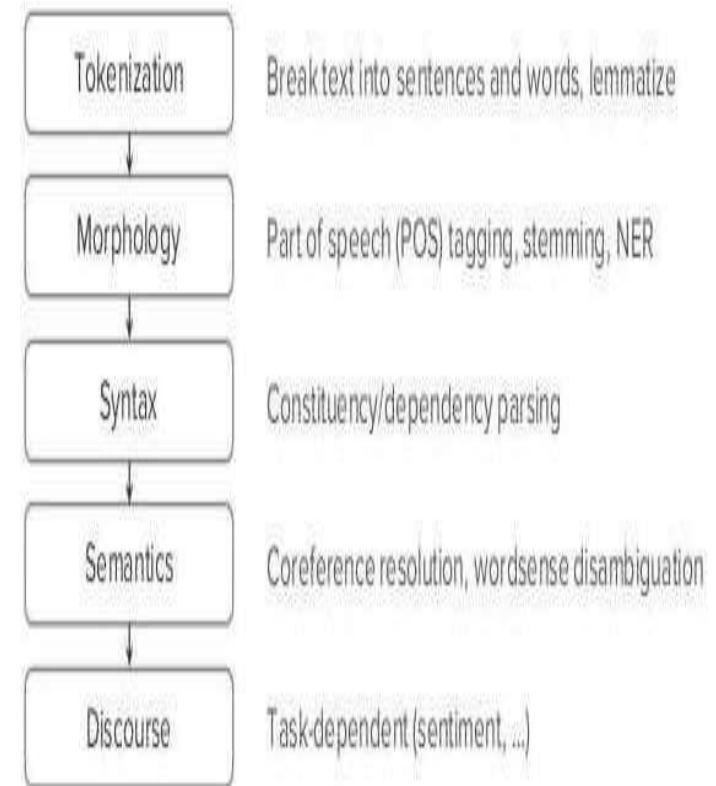
M, Vrushali, John,.....

here  $L \rightarrow \{a..z\}$

$L1 \rightarrow \{A..Z\}$

## 2. Morphological Analysis / Parsing

### “Classical” NLP Pipeline



# Morphological Analysis / Parsing

- What is a “morpheme”?

A minimal meaning-bearing unit in a language

eg. Dog -> 1 morpheme ‘dog’

cats -> 2 morphemes ‘cat’ + ‘s’

- The problem of recognizing that a word breaks down into component morphemes and building a structural representation of this fact is known as “Morphological Analysis / Parsing”

eg. doctors – doctor + s

foxes – fox + es

talking – talk + ing

(surface / input forms)    (stem)    (suffix)



- Different surfaces can have same stem

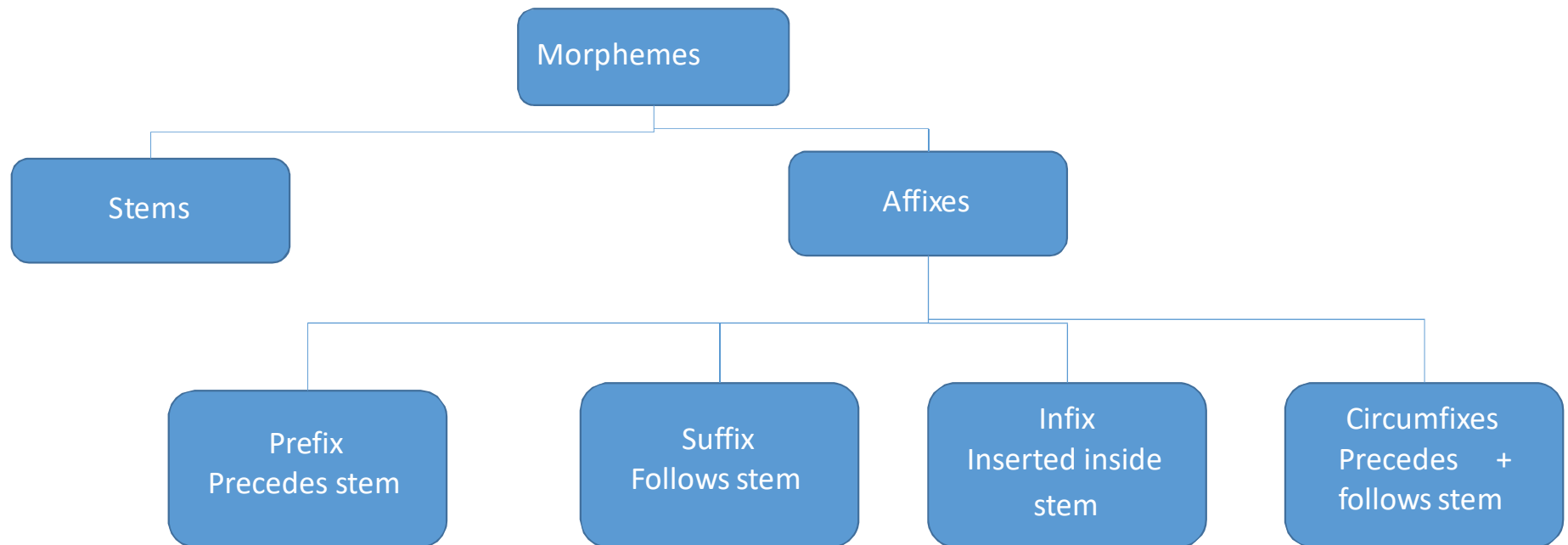
eg. sings, sang, sung - sing

surfaces

stem / common lemma

- Mapping from surfaces to stem / lemma is also called “Lemmatization”
- Stemming – simple to implement, useful in applications like IR and Web search
- Lemmatization – complex to implement, NLP applications which relies on meaning
- Can the problem of morphological analysis be solved by Lookup?
  - may be possible for a small subset but not for entire language
  - 1 verb – various forms
  - 1 noun – various forms
  - Some languages are morphologically complex – Russian, Turkish

# Morphemes



More than one affixes in the same word are possible  
Eg. Rewrites, preprocessing

# Methods / Approaches to combine morphemes to form words

- **Inflection** – combination of a word stem with a grammatical morpheme, commonly plural with ‘s’ or past tense with ‘ed’, resulting in a word of same class

eg. Papers, walked

- **Derivation** – combination of a word stem with a grammatical morpheme resulting in a word of different class
- eg. fish → fishery, computerize → computerization
- **Compounding** – combination of multiple word stem together eg. Battlefield, doghouse
- **Cliticization** – combination of a word stem with a “clitic”  
eg – I’ve                      I’ll I’m

A clitic is a morpheme that acts syntactically like a word but is reduced in the form and attached to another word

# Inflectional Morphology

- In English language, only nouns, verbs and adjectives are inflected
- Nouns can be inflected in two way –
  - affix used for plural
  - affix used for possessive

|          | Regular nouns |       | Irregular nouns |      |
|----------|---------------|-------|-----------------|------|
| Singular | cat           | box   | mouse           | ox   |
| Plural   | cats          | boxes | mice            | oxen |

Possessive suffix is realized by apostrophe + s

Eg for singular noun – child's  
for plural – students'

# Verbs

- Verbal inflection in English language is more complex
- Regular and irregular classes of verbs
- Majority of the verbs are in regular class

... as follows.

| Morphological Class         | Regularly Inflected Verbs |         |        |         |
|-----------------------------|---------------------------|---------|--------|---------|
| stem                        | walk                      | merge   | try    | map     |
| -s form                     | walks                     | merges  | tries  | maps    |
| -ing participle             | walking                   | merging | trying | mapping |
| Past form or -ed participle | walked                    | merged  | tried  | mapped  |

- While adding suffixes, spelling changes are taking place at morpheme boundaries.
- Eg beg – begged  
toss – tosses

| Morphological Class | Irregularly Inflected Verbs |          |         |
|---------------------|-----------------------------|----------|---------|
| stem                | eat                         | catch    | cut     |
| -s form             | eats                        | catches  | cuts    |
| -ing participle     | eating                      | catching | cutting |
| preterite           | ate                         | caught   | cut     |
| past participle     | eaten                       | caught   | cut     |

# Derivational Morphology

- It is a combination of a word stem with a grammatical morpheme, resulting in a word of different class.
- A common derivation in English is formation of new noun, from verb or adjective – the process is called as “nominalization”

| Suffix                                      | Base verb / adj  | Derived noun |
|---------------------------------------------|------------------|--------------|
| -ee                                         | Appoint (v)      | Appointee    |
| -er                                         | Kill (v)         | Killer       |
| -ness                                       | Fuzzy (a)        | Fuzziness    |
| Adjectives can be derived from word or noun |                  |              |
|                                             | Base Noun / Verb | Derived Adj  |
| -able                                       | Suit (v)         | Suitable     |
| -less                                       | Clue(n)          | clueless     |

# Cliticization

| Full Form | Clitic | Full Form | Clitic |
|-----------|--------|-----------|--------|
| am        | 'm     | have      | 've    |
| are       | 're    | has       | 's     |
| is        | 's     | had       | 'd     |
| will      | 'll    | would     | 'd     |

... she's can mean *she is* (

- Clitics preceding a word are Proclitics
- Clitics following a word are Enclitics
- These are rarely found in English

## Finite state morphological analysis / parsing

- What is expected in morphological analysis?

| Input   | Morphological Parse |                              |
|---------|---------------------|------------------------------|
| Cats    | Cat + N + Pl        |                              |
| Cities  | City + N + Pl       |                              |
| Caught  |                     | ◀ Catch + V + PastParticiple |
| Dog     | Dog + N + SG        |                              |
| Playing | Play +V + PresPart  |                              |

ambiguity



# How to build a Morphological parser?

- **Lexicon**

Lexicon is a list of stems and affixes along with basic information (whether a stem is a noun or verb)

- **Morphotactics**

These are guiding rules for arrangement of morphemes from different classes eg. Morphemes of which class can follow morphemes of other class

boy – boyes

girl – girlshere 'es' and 's' follow the nouns, they can not precede the noun

- **Orthographic rules**

Spelling rules – they guide about the changes occurring in a word stem while combining with other morphemes eg. City – cities .....not citys

## Exercise

- Represent morphological parse of following words.

| Input    | Morphological Parse |  |
|----------|---------------------|--|
| patents  |                     |  |
| balloons |                     |  |
| watched  |                     |  |
| animal   |                     |  |
| going    |                     |  |

- Morphological parse

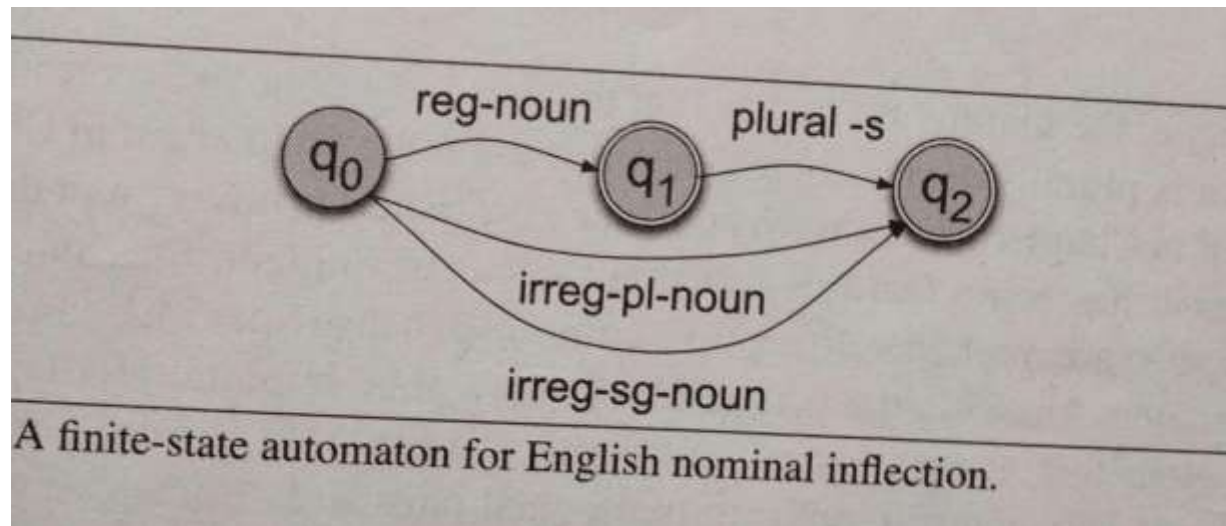
| Input    | Morphological Parse |  |
|----------|---------------------|--|
| patents  | patent + N + Pl     |  |
| balloons | balloon + N + Pl    |  |
| watched  | watch + V + Past    |  |
| animal   | animal + N + SG     |  |
| going    | go +V + PresPart    |  |

## Construction of a Finite-State lexicon

- Simplest possible Lexicon –  
every possible word in a language + proper names  
{apple, boxes, cat, ....., go, run, stop, running, gone.....Ameya,  
Bharati, ..... }

Impossible task

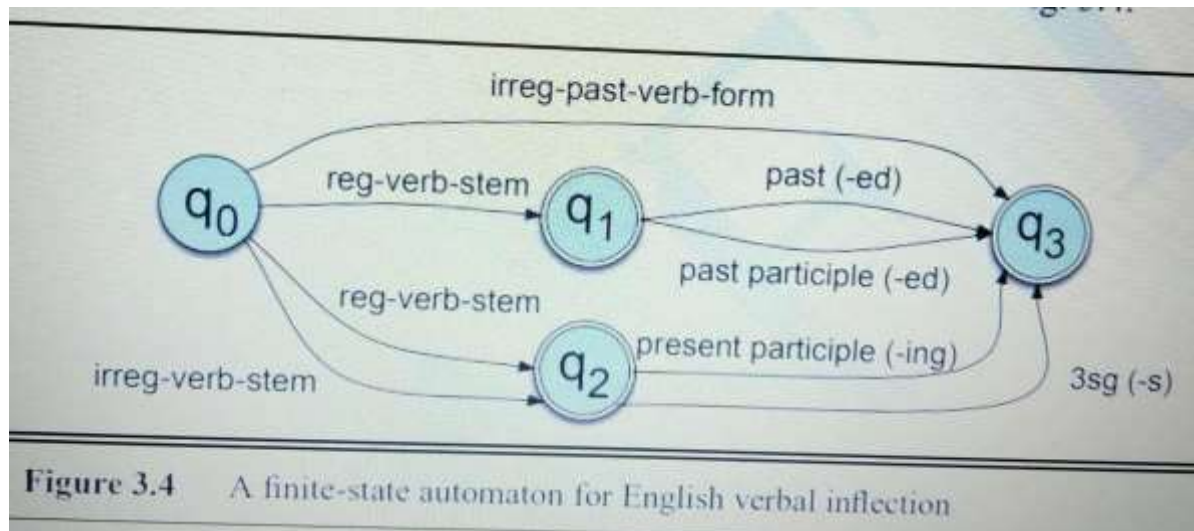
Hence, Lexicons are built with the list of each of the stems and affixes of the language together with the representation of morphotactics



Eg. Cat, dog, ....majority of nouns fall under reg-noun category.  
Ignore plurals with -es

| Reg-noun | Irreg-Pl-noun | Irreg-SG-noun | Plural -s |
|----------|---------------|---------------|-----------|
| Cat      | Geese         | Goose         |           |
| Dog      | Sheep         | Sheep         |           |
| Girl     | Mice          | mouse         |           |

## English verbal Inflection



3 forms of verb stem class

–

Regular verb stem

Irregular verb stem

Irregular past verb form

Affix class –

-ed past

-ed participle

-ing participle

-s Third singulars

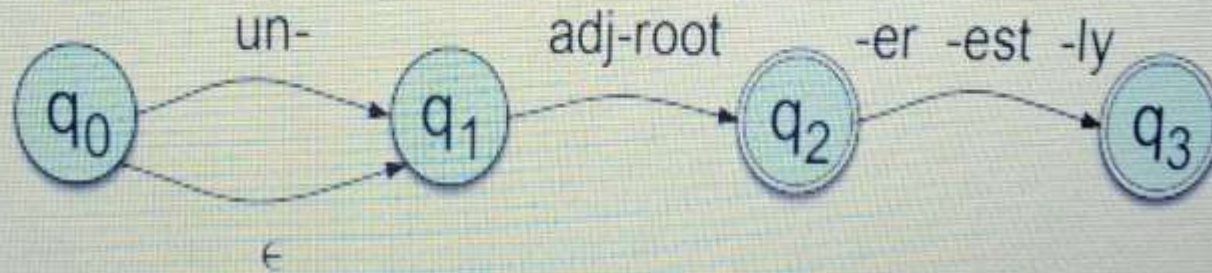
| Reg-verb-stem | Irreg-verb-stem | Irreg-past-stem | Past –ed | PastPart –ed | Pres Part –ing | Third singula r -s |
|---------------|-----------------|-----------------|----------|--------------|----------------|--------------------|
| walk          | cut             | caught          |          |              |                |                    |
| fly           | speak           | ate             |          |              |                |                    |
| talk          | sing            | sang            |          |              |                |                    |

# Derivational Morphology

Simple ex for morphotactics of English adjectives Big – bigger, biggest

Happy – happier, happiest, happily, unhappy, unhappily

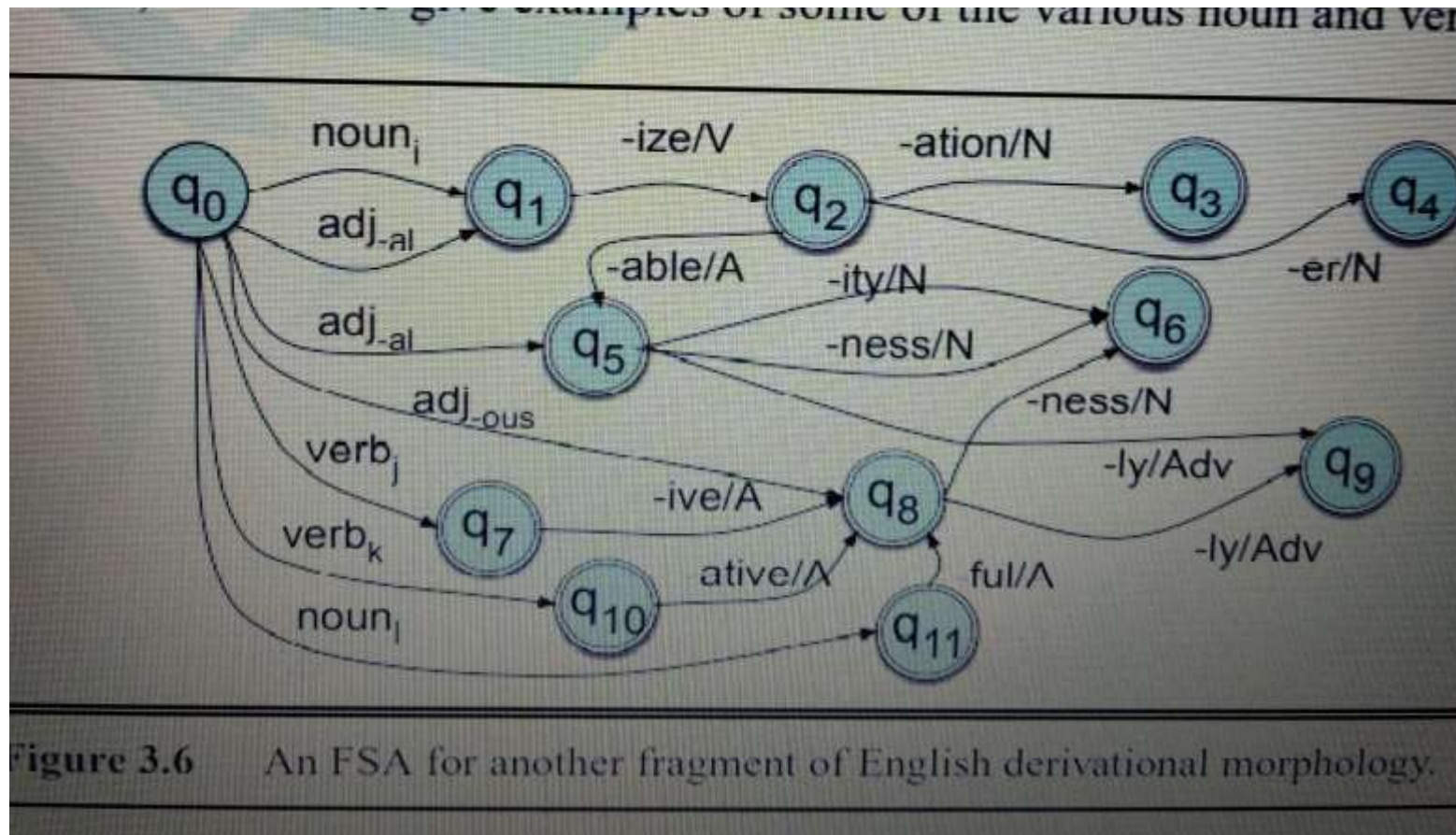
Cool – cooler, coolest, coolly



**Figure 3.5** An FSA for a fragment of English adjective morphology: Antworth's Proposal #1.



# Exercise

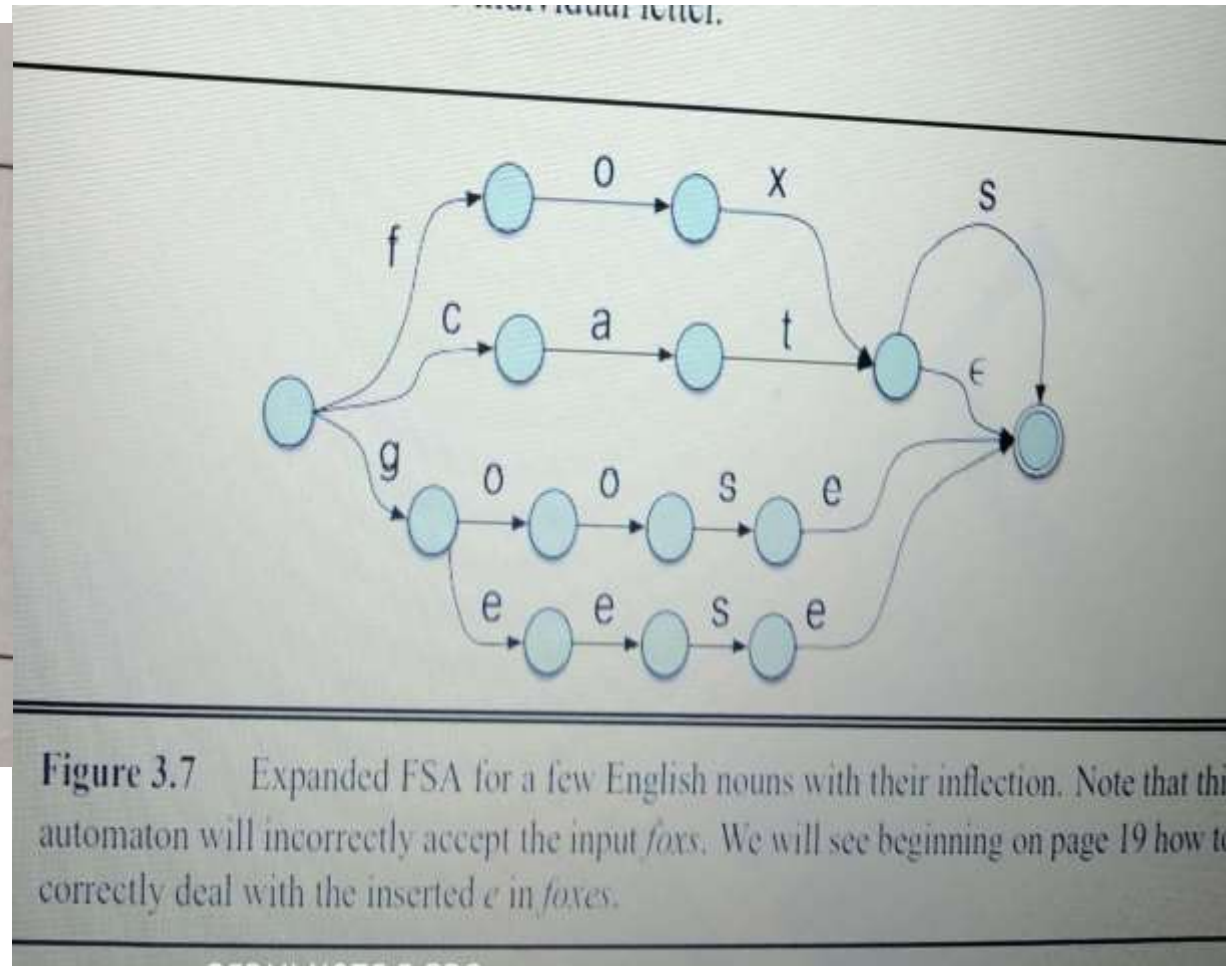
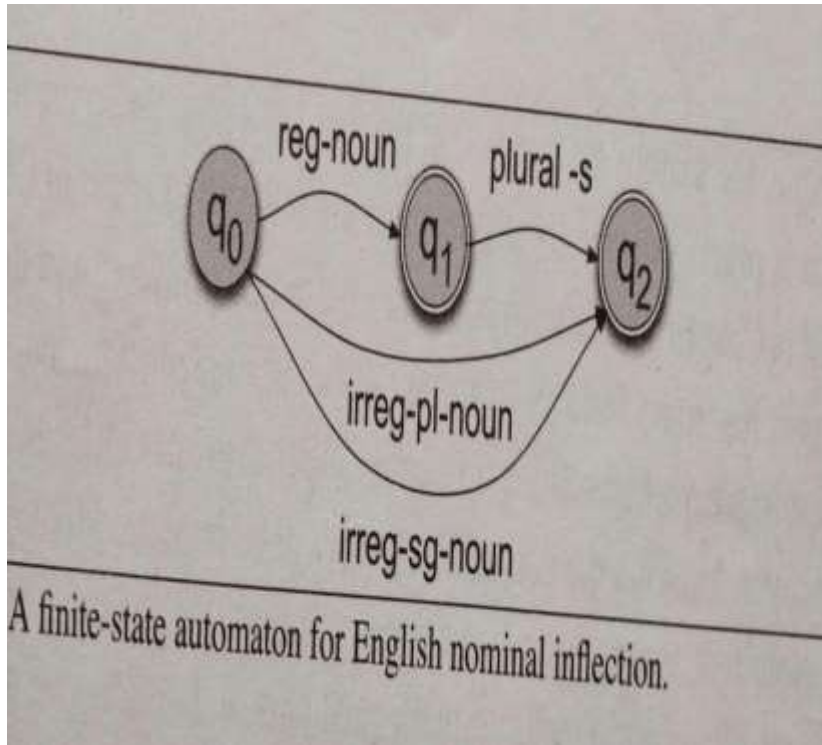




# Finite-State Morphological Parsing

- We can now use these FSAs to solve the problem of **morphological recognition**:
  - Determining whether an input string of letters makes up a legitimate English word or not
  - We do this by taking the morphotactic FSAs, and plugging in each “sub-lexicon” into the FSA.
  - The resulting FSA can then be defined as the level of the individual letter.

Morphological recognition – plug sub-lexicon into FSA  
 ie. Expand each arc like reg-noun, irreg-pl-noun etc.

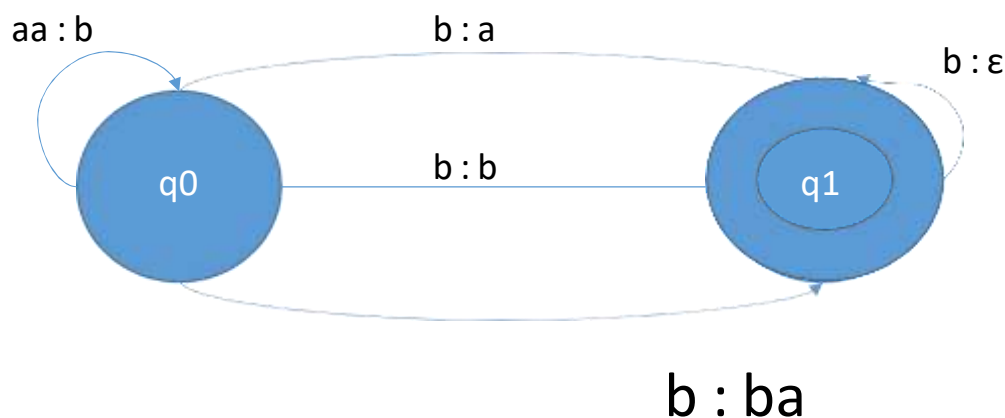


# Finite-State Transducer (FST)

- A finite-state transducer (FST) is a finite-state machine with two memory *tapes*, following the terminology for Turing machines: an input tape and an output tape.
- This contrasts with an ordinary finite-state automaton, which has a single tape.
- An FST is a type of finite-state automaton (FSA) that maps between two sets of symbols.
- An FST is more general than an FSA.
- An FSA defines a formal language by defining a set of accepted strings, while an FST defines relations between sets of strings.

# Finite State Transducer (FST)

- FST is a type of finite automata which maps between two sets of symbols
- In FST, each arc is labeled by an input and output string separated by colon



1. FST as recognizer
2. FST as generator
3. FST as translator
4. FST as set relator

FST as a morphological parser will take as input a string of letters and generate as output a string of morphemes



## FST representation

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $Q$             | a finite set of $N$ states $q_0, q_1, \dots, q_{N-1}$                                                                                                                                                                                                                                                                                                                                                                              |
| $\Sigma$        | a finite set corresponding to the input alphabet                                                                                                                                                                                                                                                                                                                                                                                   |
| $\Delta$        | a finite set corresponding to the output alphabet                                                                                                                                                                                                                                                                                                                                                                                  |
| $q_0 \in Q$     | the start state                                                                                                                                                                                                                                                                                                                                                                                                                    |
| $F \subseteq Q$ | the set of final states                                                                                                                                                                                                                                                                                                                                                                                                            |
| $\delta(q, w)$  | the transition function or transition matrix between states; Given a state $q \in Q$ and a string $w \in \Sigma^*$ , $\delta(q, w)$ returns a set of new states $Q' \subseteq Q$ . $\delta$ is thus a function from $Q \times \Sigma^*$ to $2^Q$ (because there are $2^Q$ possible subsets of $Q$ ). $\delta$ returns a set of states rather than a single state because a given input may be ambiguous in which state it maps to. |
| $\sigma(q, w)$  | the output function giving the set of possible output strings for each state and input. Given a state $q \in Q$ and a string $w \in \Sigma^*$ , $\sigma(q, w)$ gives a set of output strings, each a string $o \in \Delta^*$ . $\sigma$ is thus a function from $Q \times \Sigma^*$ to $2^{\Delta^*}$ .                                                                                                                            |

# Morphological Parsing

It is implemented by building mapping rules that maps letter sequences like

cats on the **surface** level  
into

morpheme and features sequence like cat +N +PL  
on the **lexical level**.

## FST - for morphological parsing

- For given input cats – cat + N +PI
- Here a word is represented as a correspondence between two levels – lexical level and surface level

|         |  |  |   |   |   |    |     |  |  |  |          |
|---------|--|--|---|---|---|----|-----|--|--|--|----------|
| Lexical |  |  | c | a | t | +N | +PI |  |  |  | $\Sigma$ |
| Surface |  |  | c | a | t | s  |     |  |  |  | $\Delta$ |

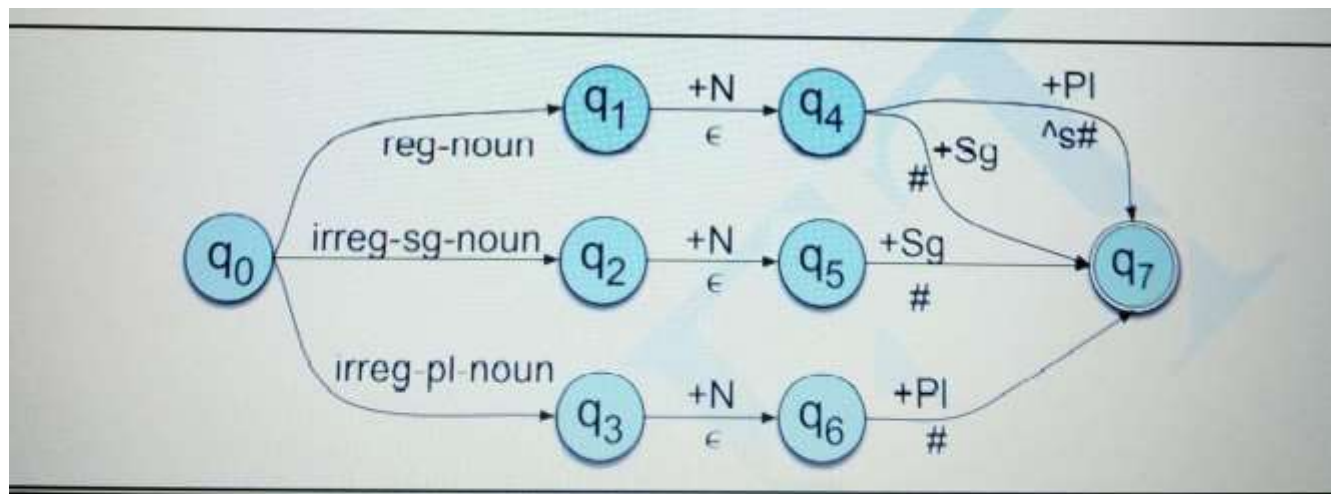
New alphabet  $\Sigma'$



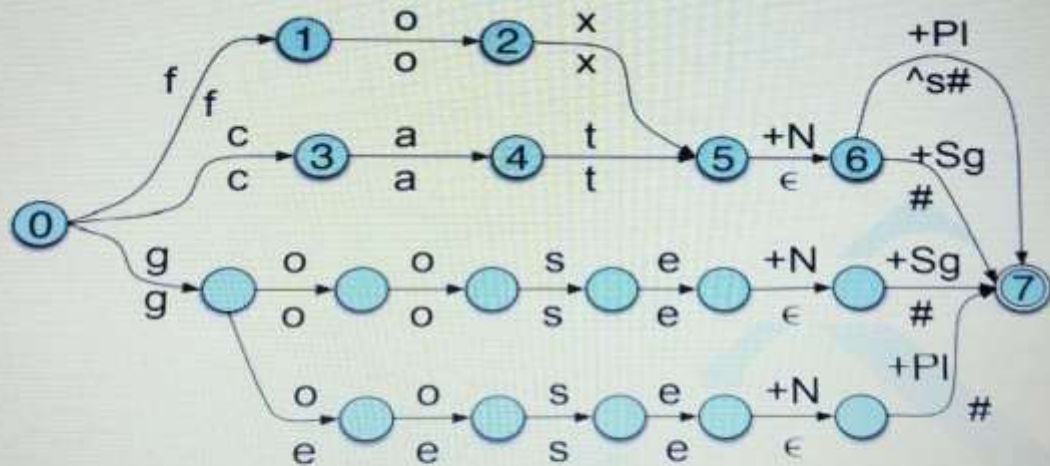
## Morphological noun parser

^ - indicates a morpheme boundary

# - indicates a word boundary



**Figure 3.13** A schematic transducer for English nominal number inflection  $T_{num}$ . The symbols above each arc represent elements of the morphological parse in the lexical tape; the symbols below each arc represent the surface tape (or the intermediate tape, to be described later), using the morpheme-boundary symbol ^ and word-boundary marker #. The labels on the arcs leaving  $q_0$  are schematic, and need to be expanded by individual

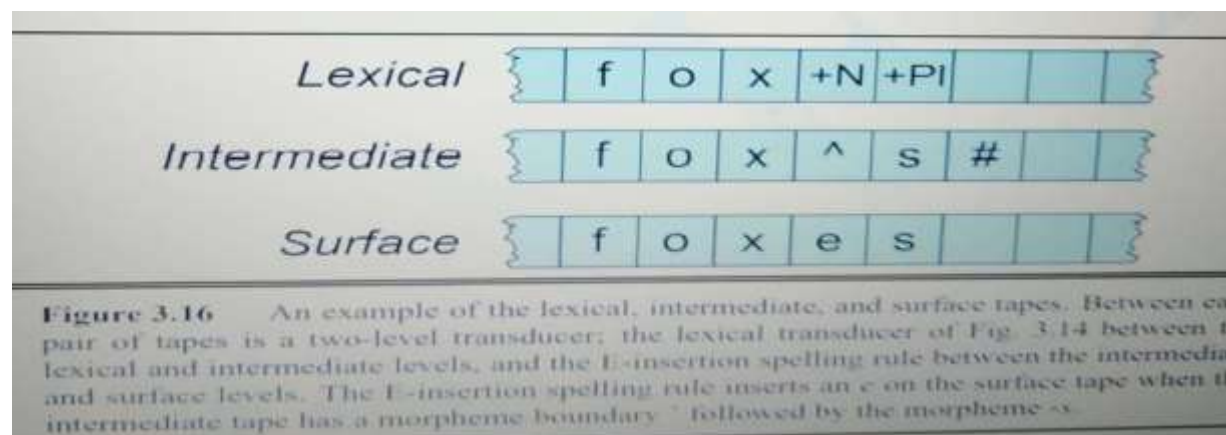


**Figure 3.14** A fleshed-out English nominal inflection FST  $T_{lex}$ , expanded from  $T_{num}$  by replacing the three arcs with individual word stems (only a few sample word stems are shown).

# Transducers and Orthographic rules

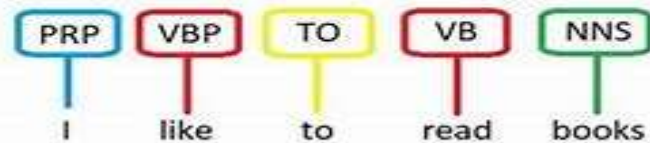
| Name               | Description of Rule                                                    | Example        |
|--------------------|------------------------------------------------------------------------|----------------|
| Consonant doubling | l-letter consonant doubled before <i>-ing/-ed</i>                      | beg/begging    |
| E deletion         | Silent e dropped before <i>-ing</i> and <i>-ed</i>                     | make/making    |
| E insertion        | e added after <i>-s, -z, -x, -ch, -sh</i> before <i>-s</i>             | watch/watches  |
| Y replacement      | <i>-y</i> changes to <i>-ie</i> before <i>-s, -i</i> before <i>-ed</i> | try/tries      |
| K insertion        | verbs ending with <i>vowel + -e</i> add <i>-k</i>                      | panic/panicked |

We can think of these spelling changes as taking as input a simple representation



# Part of Speech Tagging

## POS Tagging



# Part of Speech Tagging

- Part of Speech tagging is a process of assigning a PoS or other syntactic class marker to each word in a corpus.

(A corpus is a computer readable collection of text or speech)

- Eg Book that flight.

Book/VB that/DT flight/NN.

- Here, (Penn Treebank PoS tags)

VB – verb, base form

DT – determiner

NN – noun, singular

Popular corpora (English): Brown Corpus (1M words)  
British National Corpus – BNC (100M words)  
Wall Street Journal Corpus (30M words)

- POS is useful for subsequent syntactic parsing and word sense disambiguation

- Basic 8 parts-of-speech

noun, verb, pronoun, proposition, adverb, conjunction, participle, article

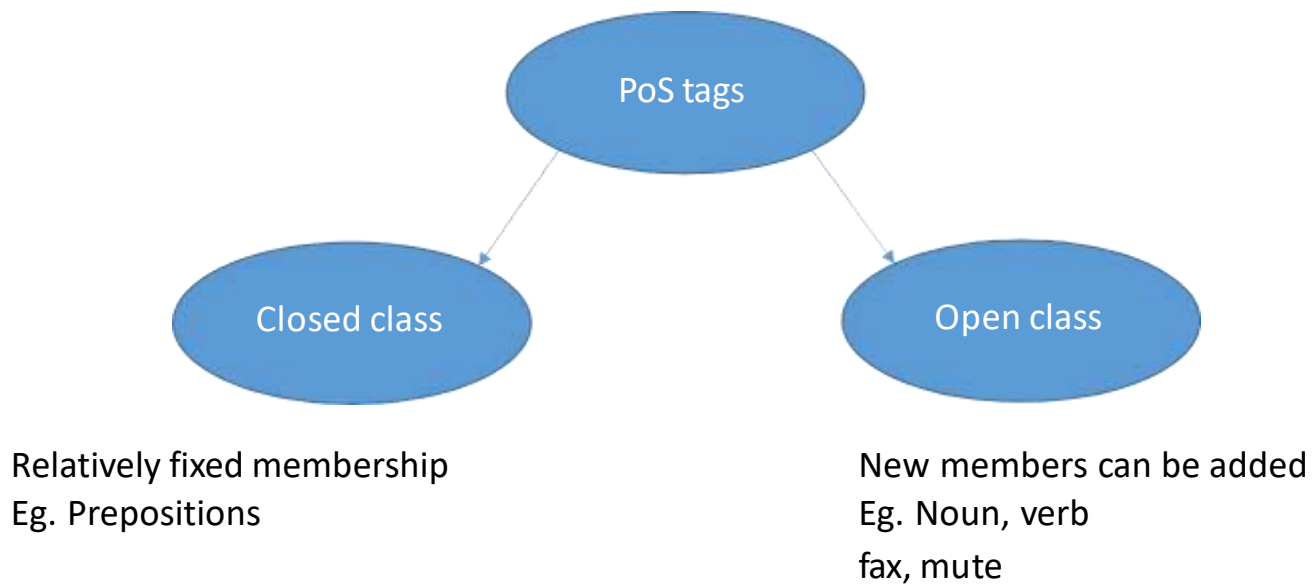
- Penn Treebank (1993) – 45 PoS tags

- Brown Corpus (1979) – 87 PoS tags

- Parts of speech are also known as word classes, morphological classes, lexical tags

- PoS tags are very useful in syntactic parsing, WSD, IR ,...

- PoS tags - closed class and open class



Part of speech is the grammatical category of a word.

- **Verb** asserts something about the subject of the sentence and express actions, events, or states of being. Ex. Walk, write, stand (main form, 3<sup>rd</sup> person SG, past participle, ...)
- **Noun** is used to name a person, an animal, a place, a thing, or an abstract idea.  
Ex. Rama, book, John (common noun, proper noun)
- **Adjective** describes nouns and pronouns by some property or quality.  
Ex. Beautiful, noisy
- **Adverb** describes a verb, a phrase, or a clause. An adverb indicates manner, time, place, cause, ... Ex. Slowly, gently (directional, locative, degree, manner, temporal)
- **Pronoun** replaces nouns or another pronouns and are essentially used to make sentences less cumbersome and less repetitive. Ex. He, she, it
- Prepositions, Conjunctions, Determiners ...



## Closed classes in English

- Prepositions – on, under, over, from, to
- Determiners – a, an, the
- Pronouns – she, I, who
- Conjunctions – and, but, or, if
- Auxiliary verbs – can, may, should
- Particles – up, down, on, off, in, out, at, by
- Numerals – one, two, first, forth

# TagSets

The process of classifying words into their parts of speech and labeling them accordingly is known as part-of-speech tagging, POS-tagging, or simply tagging. Parts of speech are also known as word classes or lexical categories. **The collection of tags used for a particular task** is known as a tagset.

# Tagsets for English

- Origin of all Tagsets for English language is “87-tag tagset” used for Brown Corpus in 1979 – research at Brown university

It is a million word collection of samples from 500 written texts from different genres  
(news papers, novels, academics, non-fiction, etc)

## Small 45-tag Penn Treebank Tagset

2. Tagsets for English 9

| Tag  | Description           | Example                | Tag | Description           | Example              |
|------|-----------------------|------------------------|-----|-----------------------|----------------------|
| CC   | Coordin. Conjunction  | <i>and, but, or</i>    | SYM | Symbol                | <i>+, %, &amp;</i>   |
| CD   | Cardinal number       | <i>one, two, three</i> | TO  | "to"                  | <i>to</i>            |
| DT   | Determiner            | <i>a, the</i>          | UH  | Interjection          | <i>ah, oops</i>      |
| EX   | Existential 'there'   | <i>there</i>           | VB  | Verb, base form       | <i>eat</i>           |
| FW   | Foreign word          | <i>mea culpa</i>       | VBD | Verb, past tense      | <i>ate</i>           |
| IN   | Preposition/sub-conj  | <i>of, in, by</i>      | VBG | Verb, gerund          | <i>eating</i>        |
| JJ   | Adjective             | <i>yellow</i>          | VBN | Verb, past participle | <i>eaten</i>         |
| JJR  | Adj., comparative     | <i>bigger</i>          | VBP | Verb, non-3sg pres    | <i>eat</i>           |
| JJS  | Adj., superlative     | <i>wildest</i>         | VBZ | Verb, 3sg pres        | <i>eats</i>          |
| LS   | List item marker      | <i>1, 2, One</i>       | WDT | Wh-determiner         | <i>which, that</i>   |
| MD   | Modal                 | <i>can, should</i>     | WP  | Wh-pronoun            | <i>what, who</i>     |
| NN   | Noun, sing. or mass   | <i>llama</i>           | WPS | Possessive wh-        | <i>whose</i>         |
| NNS  | Noun, plural          | <i>llamas</i>          | WRB | Wh-adverb             | <i>how, where</i>    |
| NNP  | Proper noun, singular | <i>IBM</i>             | \$  | Dollar sign           | <i>\$</i>            |
| NNPS | Proper noun, plural   | <i>Carolinas</i>       | #   | Pound sign            | <i>#</i>             |
| PDT  | Predeterminer         | <i>all, both</i>       | "   | Left quote            | <i>' or "</i>        |
| POS  | Possessive ending     | <i>'s</i>              | "   | Right quote           | <i>' or "</i>        |
| PRP  | Personal pronoun      | <i>I, you, he</i>      | (   | Left parenthesis      | <i>[, (, {, &lt;</i> |
| PRPS | Possessive pronoun    | <i>your, one's</i>     | )   | Right parenthesis     | <i>], ), }, &gt;</i> |
| RB   | Adverb                | <i>quickly, never</i>  | ,   | Comma                 | <i>,</i>             |
| RBR  | Adverb, comparative   | <i>faster</i>          | .   | Sentence-final punc   | <i>! ?</i>           |
| RBS  | Adverb, superlative   | <i>fastest</i>         | :   | Mid-sentence punc     | <i>: ; ... --</i>    |
| RP   | Particle              | <i>up, off</i>         |     |                       |                      |

**Figure 5.6** Penn Treebank part-of-speech tags (including punctuation).

## Exercise

A part-of-speech tagger is a piece of software that reads text in some language and decides category.

2. Tagsets for English 9

| Tag  | Description           | Example                | Tag | Description           | Example              |
|------|-----------------------|------------------------|-----|-----------------------|----------------------|
| CC   | Coordin. Conjunction  | <i>and, but, or</i>    | SYM | Symbol                | <i>+, %, &amp;</i>   |
| CD   | Cardinal number       | <i>one, two, three</i> | TO  | "to"                  | <i>to</i>            |
| DT   | Determiner            | <i>a, the</i>          | UH  | Interjection          | <i>ah, oops</i>      |
| EX   | Existential 'there'   | <i>there</i>           | VB  | Verb, base form       | <i>eat</i>           |
| FW   | Foreign word          | <i>mea culpa</i>       | VBD | Verb, past tense      | <i>ate</i>           |
| IN   | Preposition/sub-conj  | <i>of, in, by</i>      | VBG | Verb, gerund          | <i>eating</i>        |
| JJ   | Adjective             | <i>yellow</i>          | VCN | Verb, past participle | <i>eaten</i>         |
| JJR  | Adj., comparative     | <i>bigger</i>          | VBP | Verb, non-3sg pres    | <i>eat</i>           |
| JJS  | Adj., superlative     | <i>wildest</i>         | VBZ | Verb, 3sg pres        | <i>eats</i>          |
| LS   | List item marker      | <i>1, 2, One</i>       | WDT | Wh-determiner         | <i>which, that</i>   |
| MD   | Modal                 | <i>can, should</i>     | WP  | Wh-pronoun            | <i>what, who</i>     |
| NN   | Noun, sing. or mass   | <i>llama</i>           | WPS | Possessive wh-        | <i>whose</i>         |
| NNS  | Noun, plural          | <i>llamas</i>          | WRB | Wh-adverb             | <i>how, where</i>    |
| NNP  | Proper noun, singular | <i>IBM</i>             | \$  | Dollar sign           | <i>\$</i>            |
| NNPS | Proper noun, plural   | <i>Carolinas</i>       | #   | Pound sign            | <i>#</i>             |
| PDT  | Predeterminer         | <i>all, both</i>       | "   | Left quote            | <i>" or "</i>        |
| POS  | Possessive ending     | <i>'s</i>              | "   | Right quote           | <i>" or "</i>        |
| PRP  | Personal pronoun      | <i>I, you, he</i>      | (   | Left parenthesis      | <i>[, (, {, &lt;</i> |
| PRPS | Possessive pronoun    | <i>your, one's</i>     | )   | Right parenthesis     | <i>], ), }, &gt;</i> |
| RB   | Adverb                | <i>quickly, never</i>  | ,   | Comma                 | <i>,</i>             |
| RBR  | Adverb, comparative   | <i>faster</i>          | .   | Sentence-final punc   | <i>., ! ?</i>        |
| RBS  | Adverb, superlative   | <i>fastest</i>         | :   | Mid-sentence punc     | <i>: ; ... →</i>     |
| RP   | Particle              | <i>up, off</i>         |     |                       |                      |

Figure 5.6 Penn Treebank part-of-speech tags (including punctuation).

A /DT

Part-Of-Speech/NNP

Tagger/NNP

is/VBZ

a/DT

piece/NN

of/IN

software/NN

that/WDT

reads/VBZ

text/NN

in/IN

some/DT

language/NN

# Algorithms for POS tagging

- Rule based algorithms – work with a database of hand-written disambiguation rules
  - Stage 1 – assigns a potential pos tag to each word using a dictionary
    - uses 2-level transducer to return all possible pos of a word
  - Stage 2 – uses a list of hand-written disambiguation rules to narrow down this list to a single pos to each word
- Probabilistic / Stochastic taggers – use a training corpus to compute the probability of a given word having a given tag in a given context
- [The simplest stochastic taggers **disambiguate words based solely on the probability that a word occurs with a particular tag.** ]  
eg. HMM tagger



The table below shows the amount of tag ambiguity for word types in the Brown corpus

| Rule-Based Part-of-Speech Tagging |                           | 15                                             |
|-----------------------------------|---------------------------|------------------------------------------------|
|                                   | Original<br>87-tag corpus | Treebank<br>45-tag corpus                      |
| <b>Unambiguous (1 tag)</b>        | <b>44,019</b>             | <b>38,857</b>                                  |
| <b>Ambiguous (2–7 tags)</b>       | <b>5,490</b>              | <b>8844</b>                                    |
| Details:                          |                           |                                                |
| 2 tags                            | 4,967                     | 6,731                                          |
| 3 tags                            | 411                       | 1621                                           |
| 4 tags                            | 91                        | 357                                            |
| 5 tags                            | 17                        | 90                                             |
| 6 tags                            | 2 ( <i>well, beat</i> )   | 32                                             |
| 7 tags                            | 2 ( <i>still, down</i> )  | 6 ( <i>well, set, round, open, fit, down</i> ) |
| 8 tags                            |                           | 4 ( <i>'s, half, back, a</i> )                 |
| 9 tags                            |                           | 3 ( <i>that, more, in</i> )                    |

**Figure 5.10** The amount of tag ambiguity for word types in the Brown corpus, from the ICAME release of the original (87-tag) tagging and the Treebank-3 (45-tag) tagging. Numbers are not strictly comparable because only the Treebank segments 's. An earlier estimate of some of these numbers is reported in DeRose (1988).

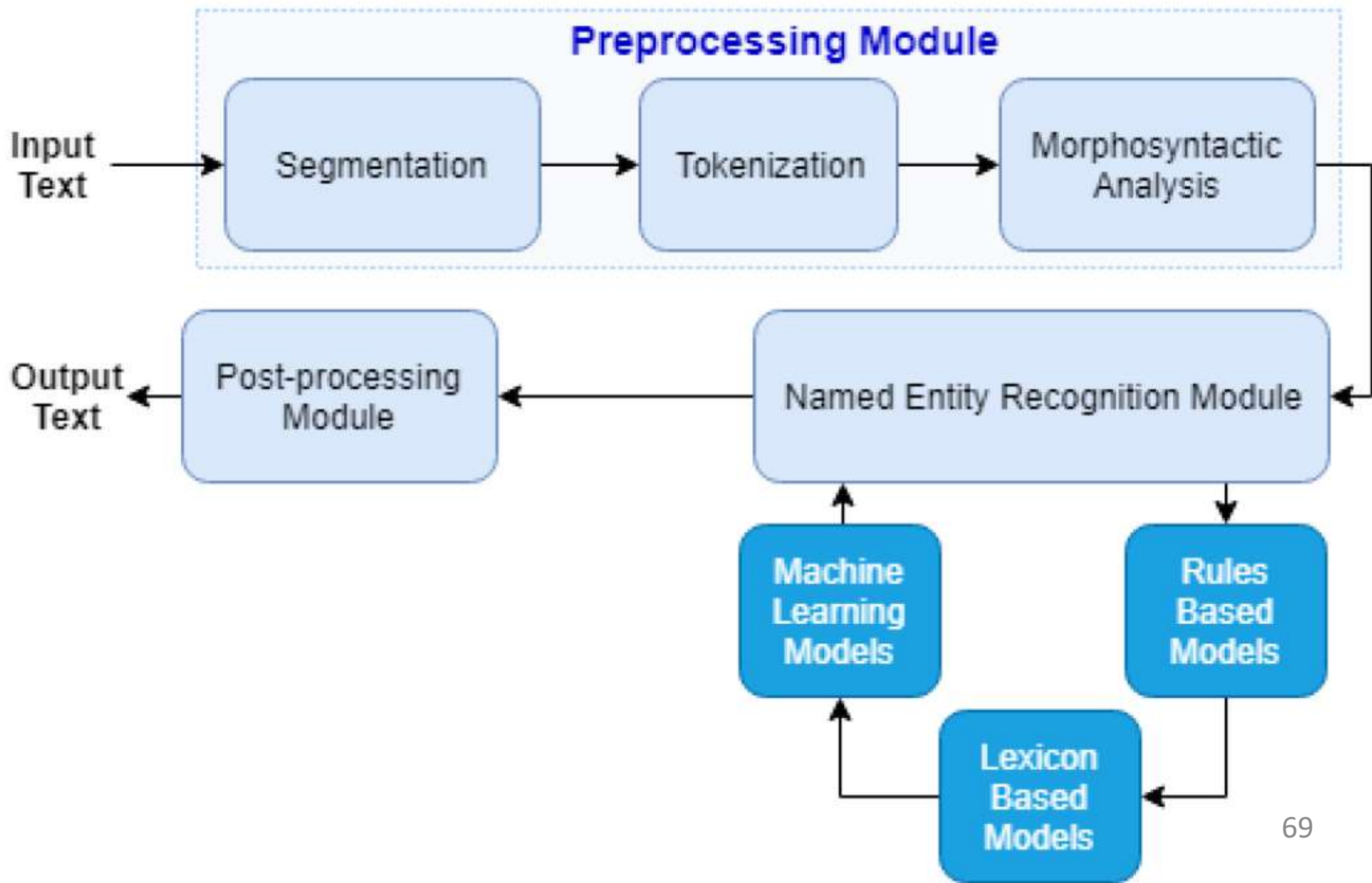


# Named Entities

**Named Entity Recognition (NER)** (also known as **(named) entity identification**, **entity chunking**, and **entity extraction**) is a subtask of [information extraction](#) that seeks to locate and classify [named entities](#) mentioned in [unstructured text](#) into pre-defined categories such as

- person names,
- organizations,
- locations,
- [medical codes](#),
- time expressions,
- quantities,
- monetary values,
- percentages, etc.

# NER



## Example of NER

Person p Loc l Org o Event e Date d Other z

Barack Hussein Obama II ✕ (born August 4, 1961 ✕) is an American ✕ attorney and politician who served as the 44th President of the United States ✕ from January 20, 2009 ✕, to January 20, 2017 ✕. A member of the Democratic Party ✕, he was the first African American ✕ to serve as president. He was previously a United States Senator ✕ from Illinois ✕ and a member of the Illinois State Senate ✕.

# Research at IITB

Browser tabs: (no subject) - vrushali.kulkarni@... | Computing for Indian Language | +

Address bar: Not secure | cfilt.iitb.ac.in





## Center For Indian Language Technology

Indian Institute of Technology, Bombay



Home

Join us

People

Downloads

News

Publications

Tools

Different Guidelines

Resources

Events

ML - Linguistics Lectures

Local

CFILT, IIT Bombay



Center for Indian Language Technology (CFILT) was set up with a generous grant from the Department of Information Technology (DIT), Ministry of Communication and Information Technology, Government of India in 2000 at the Department of Computer Science and Engineering, IIT Bombay. Prior to this the Natural Language Processing (NLP) activity of the CSE Department, IIT Bombay took off in 1996 with a grant from the United Nations University, Tokyo to create a multilingual information exchange system for the web. The project called Universal Networking Language (UNL; [www.undl.org](http://www.undl.org)) was participated in by 15 research groups across continents.

At any point of time about 30 research members work in CFILT, which includes PhD, masters and bachelor students, faculty members, linguists and lexicographers.

Deep semantics and multilinguality has throughout played a pivotal role in the activities of CFILT. The stress on semantics has led to research in the following fronts:

- **Lexical Resources:** Multilingual wordnets and ontologies and their linking
- **Lexical and Structural Disambiguation:** Resolve word and attachment ambiguities
- **Shallow Parsing:** Identifying correct parts of speech, named entities and non-recursive noun phrases for Marathi and Hindi
- **Cross Lingual Information Retrieval:** Indian language query to English and Hindi Retrieval
- **Machine Translation:** Automatic translation involving Marathi, Hindi and English
- **Text Entailment:** Testing if a piece text (hypothesis) is inferable from another (text)

Events

Prof. Pushpak Bhattacharyya's

Activate Windows

Go to Settings to activate Windows.

Show all

Taskbar: IMG\_20200728\_17...jpg | IMG\_20200728\_17...jpg | IMG\_20200728\_17...jpg | IMG\_20200728\_17...jpg | IMG\_20200728\_17...jpg | ENG 6:19 PM



Browser tabs: (no subject) - vrushali.kulkarni@... x Computing for Indian Language x +

Address bar: Not secure | cfilt.iitb.ac.in

**Prof. Pushpak in Hindustan Times News paper** NEW!

**Prof. Pushpak's interview as Faculty of Distinction** NEW!

**CFILT on Loksatta News Paper** NEW!

**5th International Sanskrit Computational Linguistics Symposium (SCLS), January-2013** NEW!

**IndoWordnet Workshop Papers** NEW!

To view papers presented at the 3rd IndoWordnet Workshop held at IIT Kharagpur, Please click [here](#)

**IndoWordnet Workshop 2010** NEW!

The 3rd IndoWordnet Workshop will be held at IIT Kharagpur, on 8th of December, 2010. The Workshop is a part of **ICON 2010** Conference.

**Global Wordnet Conference 2010**

The 5th International Conference (GWC-2010) of the Global WordNet Association (GWA) was held in IIT Bombay, Mumbai,

### Wordnets



Browse the Hindi Wordnet through [this interface](#). Inspired by the English WordNet, It is more than a conventional Hindi dictionary.



Click [here](#) to browse the Marathi WordNet. It gives different relations between synsets or synonym sets which represent unique concepts.



Browse the Sanskrit Wordnet through [this interface](#). Inspired by the English WordNet, It is more than a conventional Sanskrit dictionary.

[Brochure >>](#)



**हिंदी शब्दमित्र (Hindi Shabdmitra)** is an e-learning product meant for Hindi language teaching and learning. It uses [Hindi Wordnet](#) as a resource, which is, then, further augmented with audio-visual features, grammatical properties and is presented in a learner-friendly layered format.

### Publications



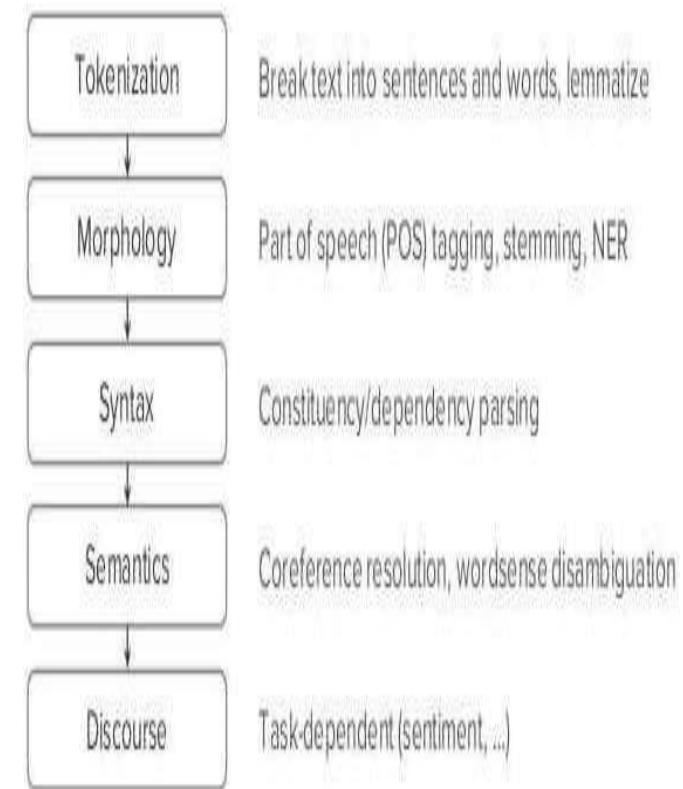

Activate Windows  
Go to Settings to activate Windows.

Show all

Taskbar: Type here to search | [Icons: File Explorer, Mail, Chrome, Word, PowerPoint, Excel] | ENG IN 6:21 PM 7/28/2020

# 3. Syntax Analysis

## “Classical” NLP Pipeline



# Parsing / Syntax Analysis

- Given a string of terminals and a CFG, determine if the string can be generated by the CFG.
  - Also return a parse tree for the string
  - return all possible parse trees for the string
- Must search space of derivations for one that derives the given string.
  - **Top-Down Parsing**: Start searching space of derivations for the start symbol.
  - **Bottom-up Parsing**: Start search space of reverse derivations from the terminal symbols in the string.



# Parsing

- Syntactic Parsing - Produce the correct syntactic parse tree for a sentence.

Context Free Grammars (CFG) =  $\{N, \Sigma, R, S\}$

- $N$  a set of non-terminal symbols (or variables)
- $\Sigma$  a set of terminal symbols (disjoint from  $N$ )
- $R$  a set of productions or rules of the form  $A \rightarrow \beta$ ,  
where  $A$  is a non-terminal and  $\beta$  is a string of symbols from  $(\Sigma \cup N)^*$
- $S$ , a designated non-terminal called the start symbol

# Example of CFG

## Grammar

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Pronoun$

$NP \rightarrow Proper-Noun$

$NP \rightarrow Det Nominal$

$Nominal \rightarrow Noun$

$Nominal \rightarrow Nominal Noun$

$Nominal \rightarrow Nominal PP$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$VP \rightarrow VP PP$

$PP \rightarrow Prep NP$

## Lexicon

Det -> the | a | that | this

Noun -> book | flight | money

Verb -> book | prefer | include

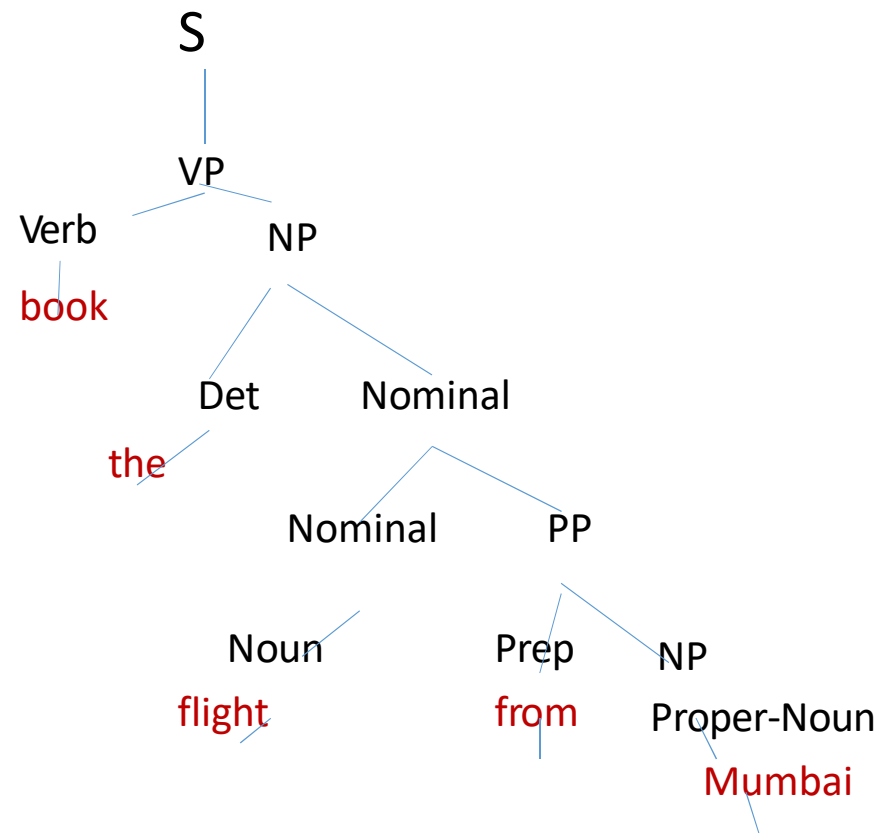
Pronoun -> I | he | she | me

Proper-Noun -> Pune | Mumbai

Aux -> does

Prep -> from | to | on | near | through

- Sentences are generated by recursively rewriting the start symbol using the productions until only terminals symbols remain.
- Ex. - book the flight from Mumbai



# The miniature English grammar and lexicon

## Grammar

S → NP VP

S → Aux NP VP

S → VP

NP → Pronoun

NP → Proper-Noun

NP → Det Nominal

Nominal → Noun

Nominal → Nominal Noun

Nominal → Nominal PP

VP → Verb

VP → Verb NP

VP → Verb NP PP

VP → Verb PP

VP → VP PP

PP → Preposition NP

## Lexicon

Det -> the a that this

Noun -> book flight money

Verb -> book prefer include

Pronoun -> I he she me we

Proper-Noun -> Pune Mumbai Hyderabad

Aux -> does

Preposition -> from to on near through

### Exercise

Parse following sentences

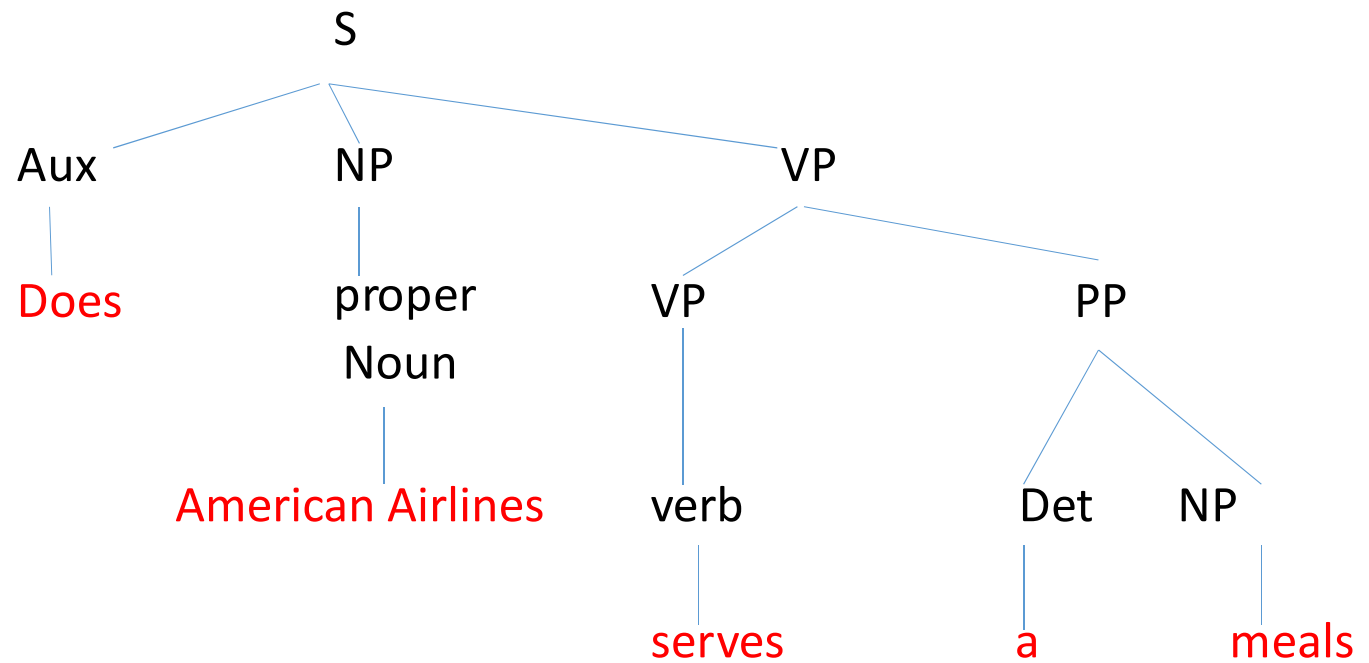
Does American airlines serves a meal?

She plays in the garden.

Top-down

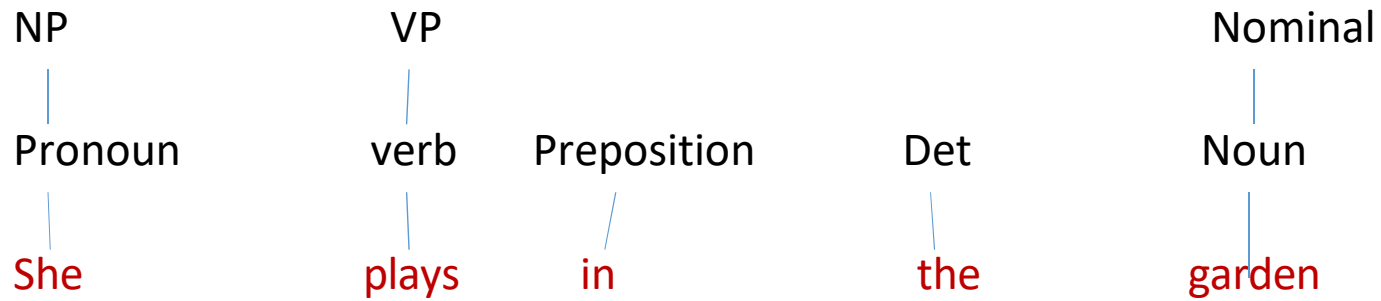
Bottom-up

Does American airlines serves a meal? (top-down parse)



She plays in the garden.

Bottom-up parse

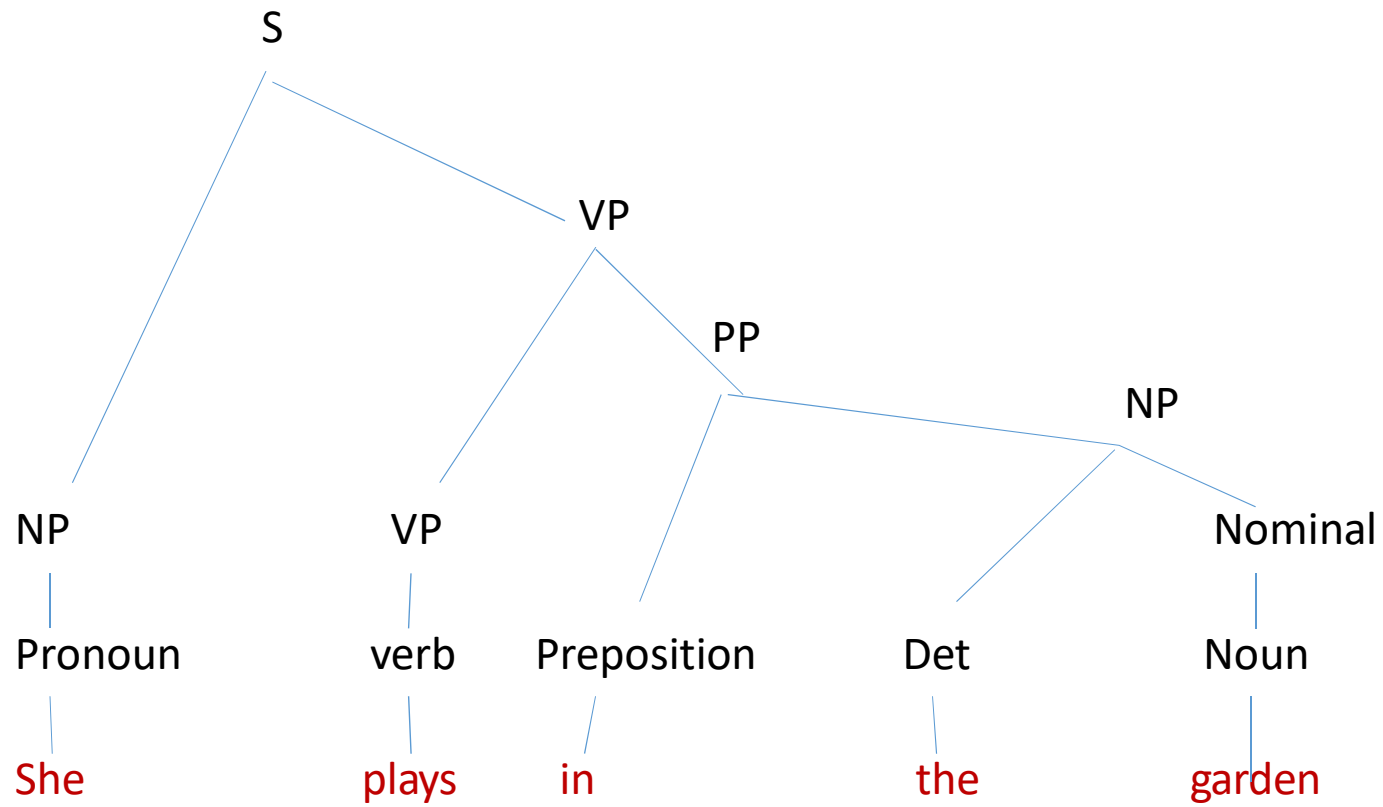


NP  
|  
Pronoun  
|  
She

VP  
|  
verb  
|  
plays

Preposition  
|  
in

NP  
/   \  
Det   Nominal  
|   |  
the   Noun  
|  
garden





# Ambiguity

- Present at various phases
- Lexical level – pos tagging  
ex. **Book** that flight. Book as Noun or Verb – Rule based algo, WSD
- Syntactic Level –  
ambiguity rises in syntactic structures – structural ambiguity multiple parse trees are possible for the same sentence
- Syntactic ambiguity
  - attachment ambiguity
  - coordination ambiguity

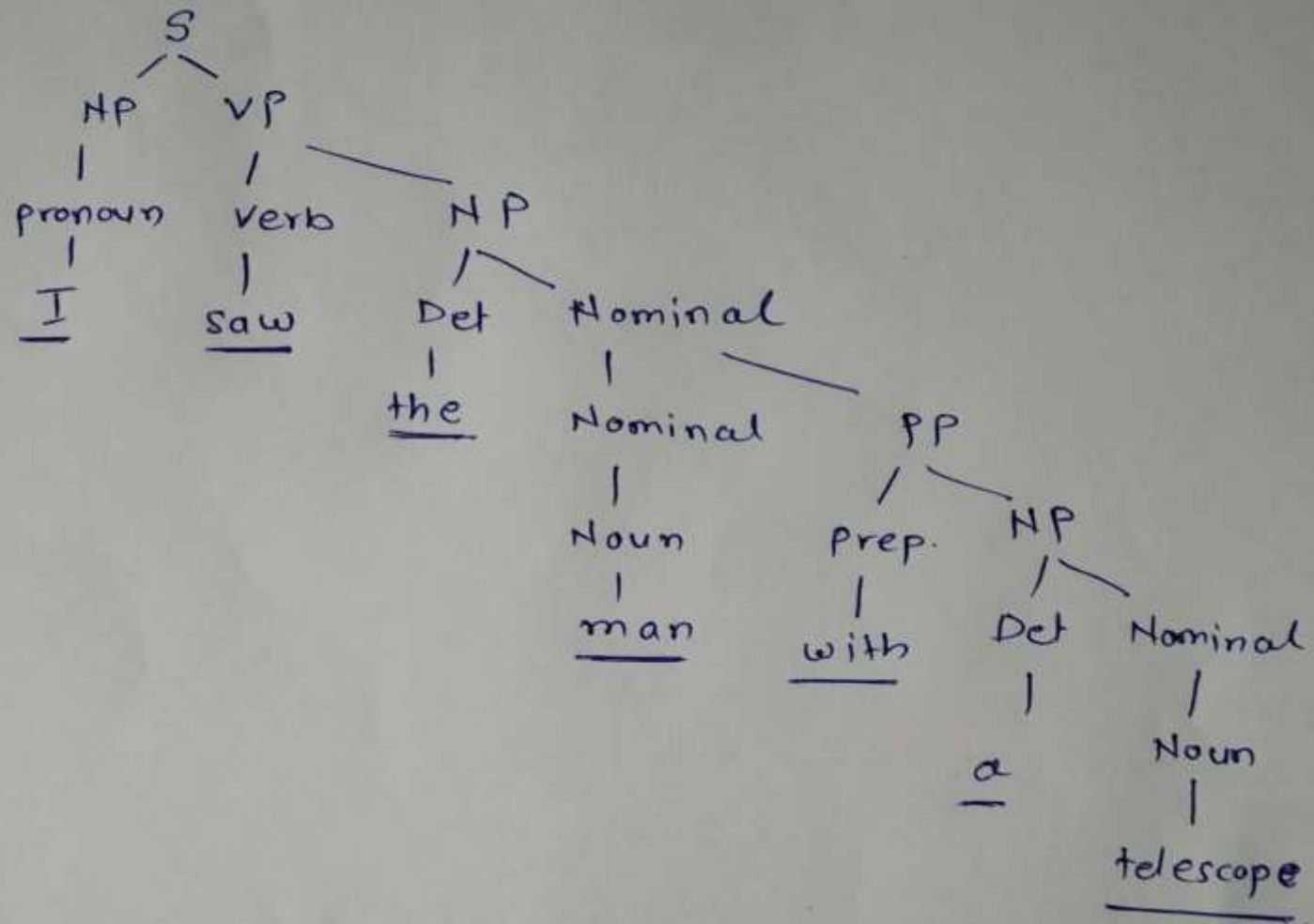
# Attachment ambiguity

- This ambiguity exists if a particular constituent can be attached to the parse tree at more than one places.

Ex. I saw the man with a telescope.

# One of the possible parse trees

ex. I saw the man with a telescope.



# Attachment ambiguity

- This ambiguity exists if a particular constituent can be attached to the parse tree at more than one places.

Ex. I saw the man with a telescope.

I saw the man with a telescope.

***“Guna ate an ice cream with fruits from Chennai”***

In this sentence, we have two prepositional phrases “***with fruits***” and “***from Chennai***”.

Here the possible meanings are as follows;

1. Guna who is from Chennai ate an ice cream filled with fruits.
2. Guna ate an ice cream filled with fruits and the ice cream is brought from Chennai.
3. Guna who is from Chennai ate the ice cream with the help of fruits.
4. Guna with the help of fruits ate the ice cream which is brought from Chennai

Here we got four possibilities due to two prepositional phrases. Each one arises from how we attach the prepositional phrases “***with fruits***” and “***from Chennai***” to either “***Guna***” or the “***ice cream***”.

I saw the man on the hill with a telescope.

I saw the man on the hill with a telescope.

I saw the man on the hill with a telescope

Attachment ambiguity arises from **uncertainty**  
**of attaching a phrase or clause to a part of a**  
**sentence**

## Coordination ambiguity

- This ambiguity exists when different sets of phrases are combined by a conjunction like “and”

Ex. Old men and women, young boys and girls

- ☐ (old men) and (women)
- ☐ old (men and women)



“delicious cookies and milk”

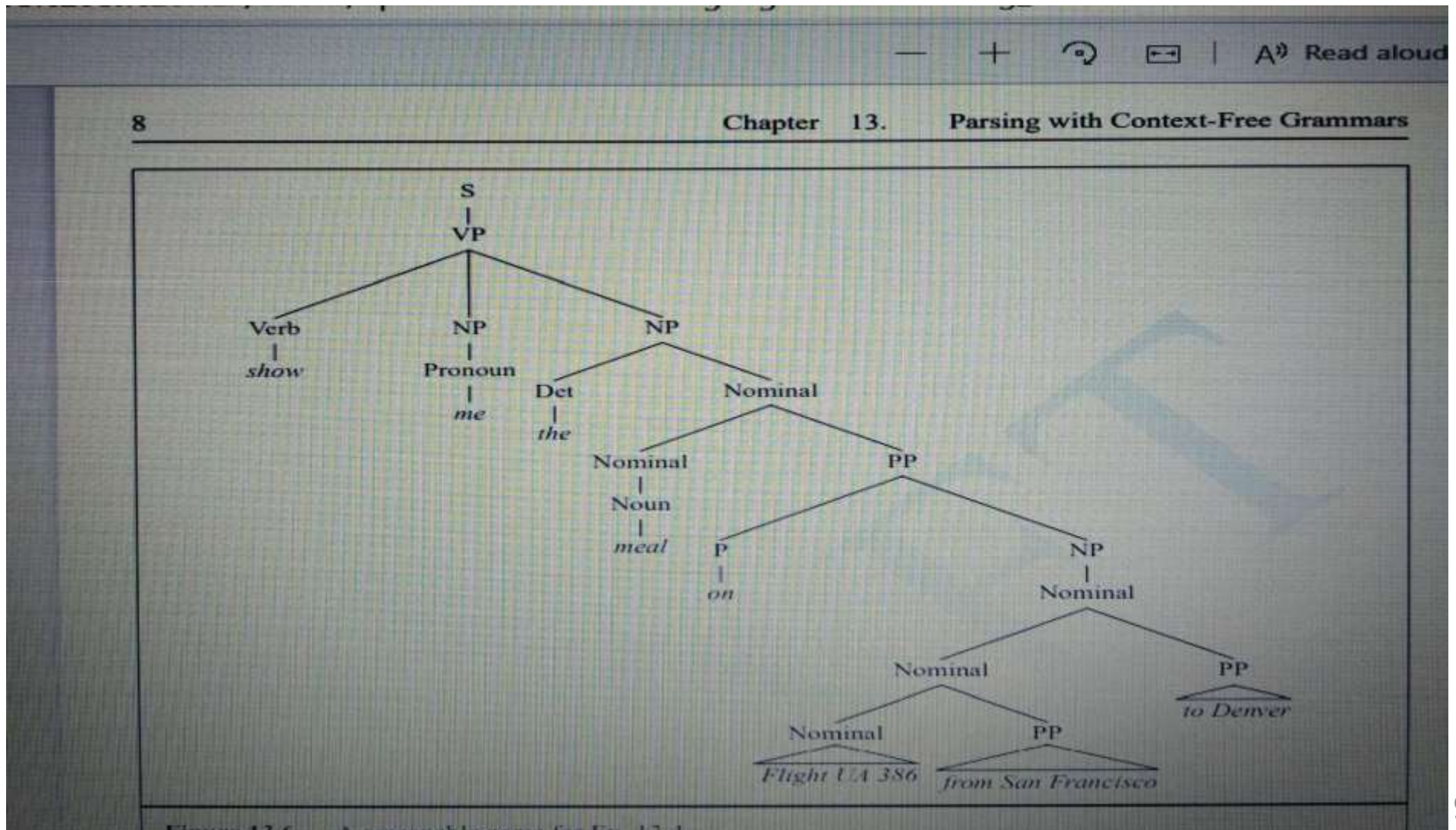
could mean

① “delicious cookies” + “milk”

Or

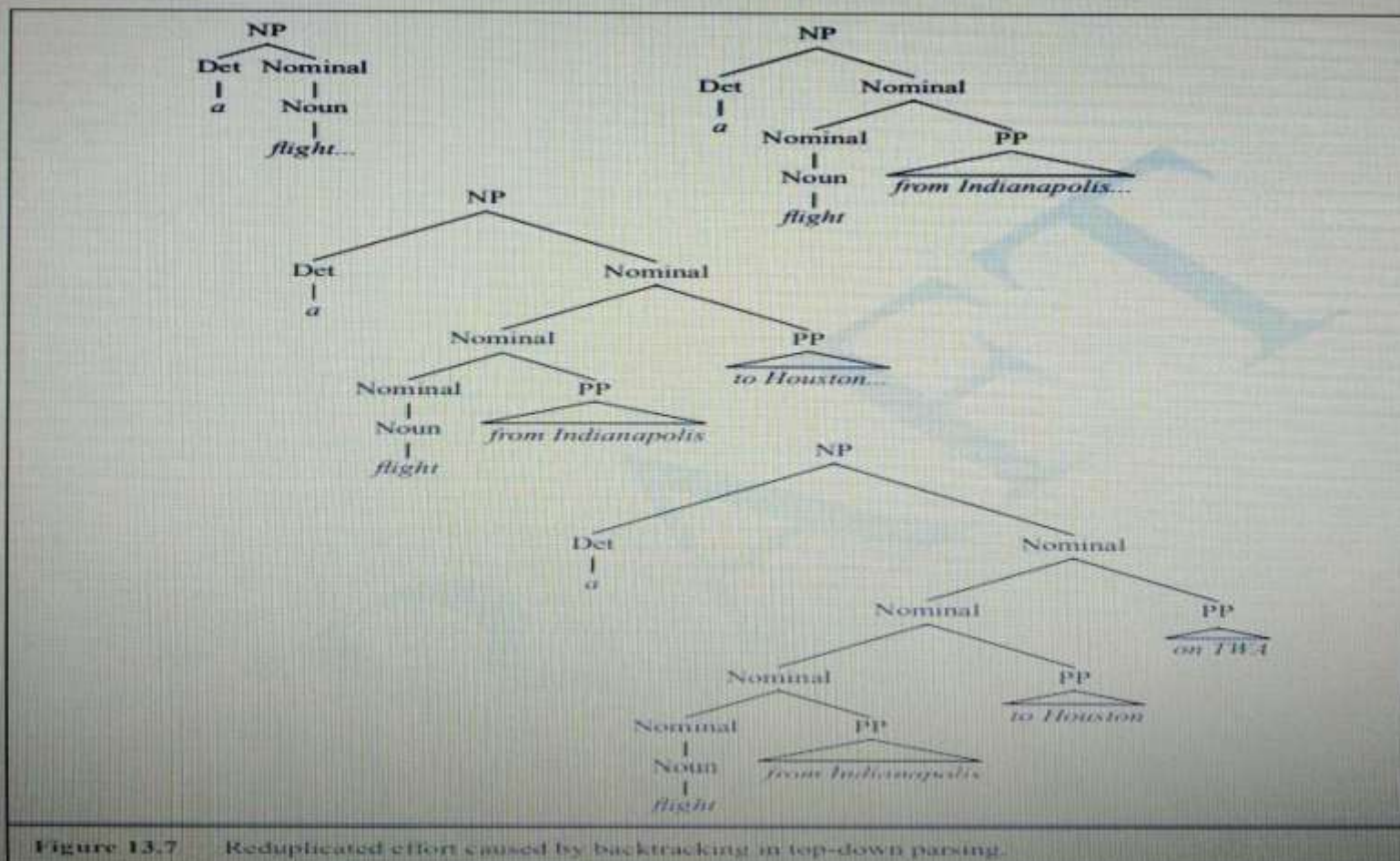
② “delicious cookies” + “delicious milk”

- Ex. Show me the meal on Flight UA 386 from San Francisco to Denver. Total 14 parse for this statement – tree below shows a reasonable parse



## Disambiguation -

- Explore all possible parse trees in parallel .... Unrealistic based on memory requirement
- Use backtracking strategy – expand the search –space tree by exploring one state at a time and backtrack if required. Inefficient as many subtrees to be rebuilt.



## EFFICIENT PARSING FOR CONTEXT FREE GRAMMAR

- Dynamic programming parsing methods

Dynamic programming is based on Principle of Optimality. It systematically solves sub-problems and stores intermediate results – no work duplication

In parsing with Dynamic programming, the subtrees are computed once, stored (in tables) and reused.

- CKY parsing – uses CNF grammar
- The Early algorithm – CFG
- Chart parsing - CFG

## What is CKY in NLP?

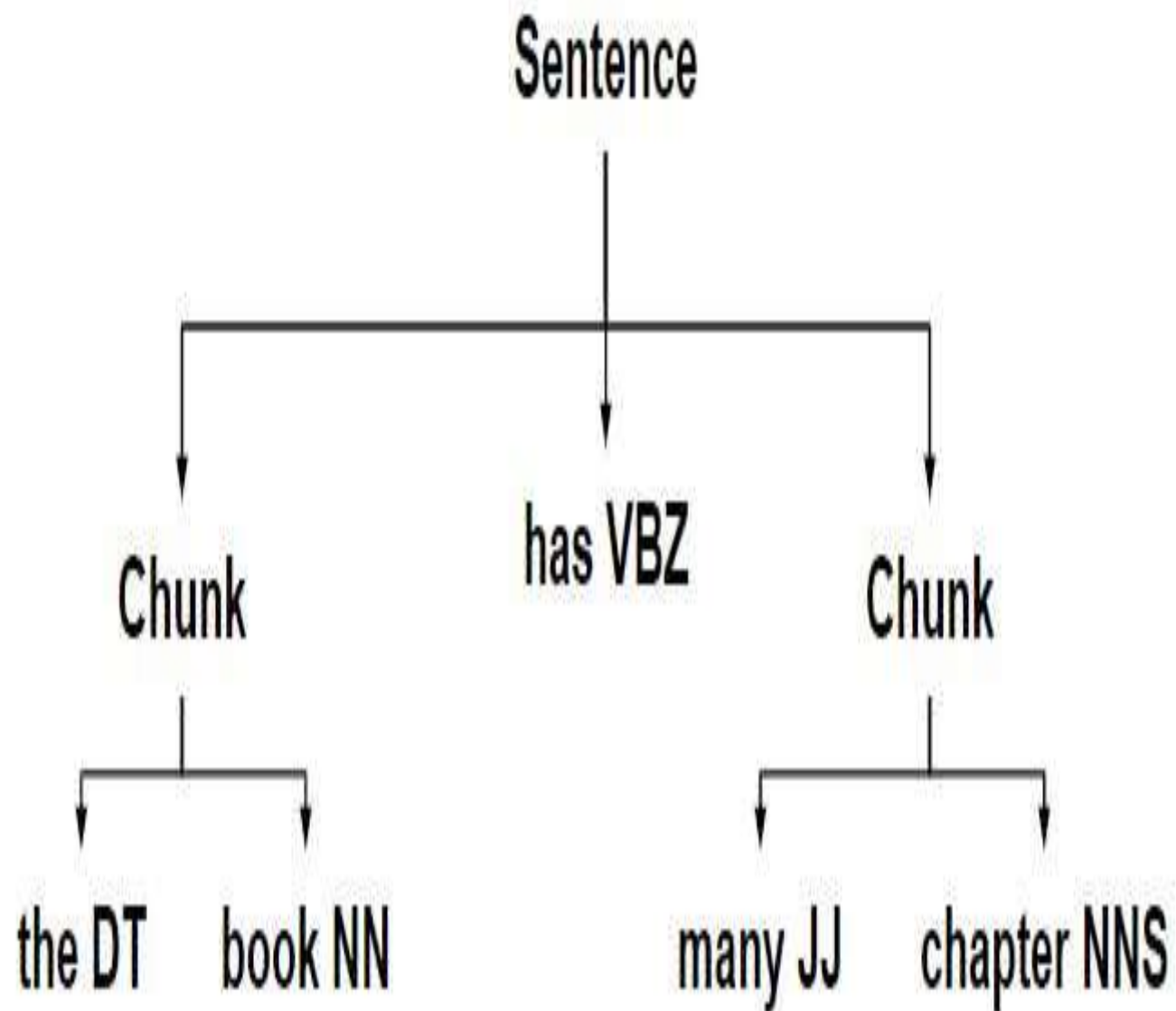
- CKY means **Cocke-Kasami-Younger**.
- It is one of the earliest recognition and parsing algorithms.
- The standard version of CKY can only recognize languages defined by context-free grammars in Chomsky Normal Form (CNF).
- In its simplest form, the CYK algorithm **solves the recognition problem; it determines whether a string can be derived from a grammar** .
- In other words, the algorithm takes a sentence and a context-free grammar and returns TRUE if there is a valid parse tree or FALSE otherwise

# Partial Parsing

- Many language processing tasks do not require complete parse trees to be generated  
.....complete parse tree is a complex process
- Partial parse or shallow parse is generated
- Ex. Information extraction, Information retrieval
- Chunking – one of the ways of performing partial parsing
  - Finite state Rule based chunking
  - Machine Learning-based chunking

- When we have loads of descriptions or modifications around a particular word or the phrase of our interest, we use chunking to grab the required phrase alone, ignoring the rest around it.
- Hence, chunking paves a way to group the required phrases and exclude all the modifiers around them which are not necessary for our analysis. Summing up, chunking helps us extract the important words alone from lengthy descriptions. Thus, chunking is a step in information extraction.





# Statistical Parsing

- Statistical / Probabilistic parsing helps to solve the problem of disambiguation
- Compute the probability of each possible interpretation and choose the most probable interpretation
- Most of the modern parsers in NLP are probabilistic
- Probabilistic parsers use Probabilistic / Stochastic CFG (PCFG)
  - augmenting CFG with probabilities
- In PCFG, each production rule will be of the form
$$A \rightarrow \beta \quad [p] \dots\dots\dots p \text{ is prob that NT } A \text{ will be expanded to } \beta$$

$p$  is a number between 0 and 1

# PCFG

## 4.1. Probabilistic Context-Free Grammars

|                                    |       |                                                          |
|------------------------------------|-------|----------------------------------------------------------|
| $S \rightarrow NP VP$              | [.80] | $Det \rightarrow that [.10] \mid a [.30] \mid the [.60]$ |
| $S \rightarrow Aux NP VP$          | [.15] | $Noun \rightarrow book [.10] \mid flight [.30]$          |
| $S \rightarrow VP$                 | [.05] | $\mid meal [.15] \mid money [.05]$                       |
| $NP \rightarrow Pronoun$           | [.35] | $\mid flights [.40] \mid dinner [.10]$                   |
| $NP \rightarrow Proper-Noun$       | [.30] | $Verb \rightarrow book [.30] \mid include [.30]$         |
| $NP \rightarrow Det Nominal$       | [.20] | $\mid prefer; [.40]$                                     |
| $NP \rightarrow Nominal$           | [.15] | $Pronoun \rightarrow I [.40] \mid she [.05]$             |
| $Nominal \rightarrow Noun$         | [.75] | $\mid me [.15] \mid you [.40]$                           |
| $Nominal \rightarrow Nominal Noun$ | [.20] | $Proper-Noun \rightarrow Houston [.60]$                  |
| $Nominal \rightarrow Nominal PP$   | [.05] | $\mid TWA [.40]$                                         |
| $VP \rightarrow Verb$              | [.35] | $Aux \rightarrow does [.60] \mid can [.40]$              |
| $VP \rightarrow Verb NP$           | [.20] | $Preposition \rightarrow from [.30] \mid to [.30]$       |
| $VP \rightarrow Verb NP PP$        | [.10] | $\mid on [.20] \mid near [.15]$                          |
| $VP \rightarrow Verb PP$           | [.15] | $\mid through [.05]$                                     |
| $VP \rightarrow Verb NP NP$        | [.05] |                                                          |
| $VP \rightarrow VP PP$             | [.15] |                                                          |
| $PP \rightarrow Preposition NP$    | [1.0] |                                                          |

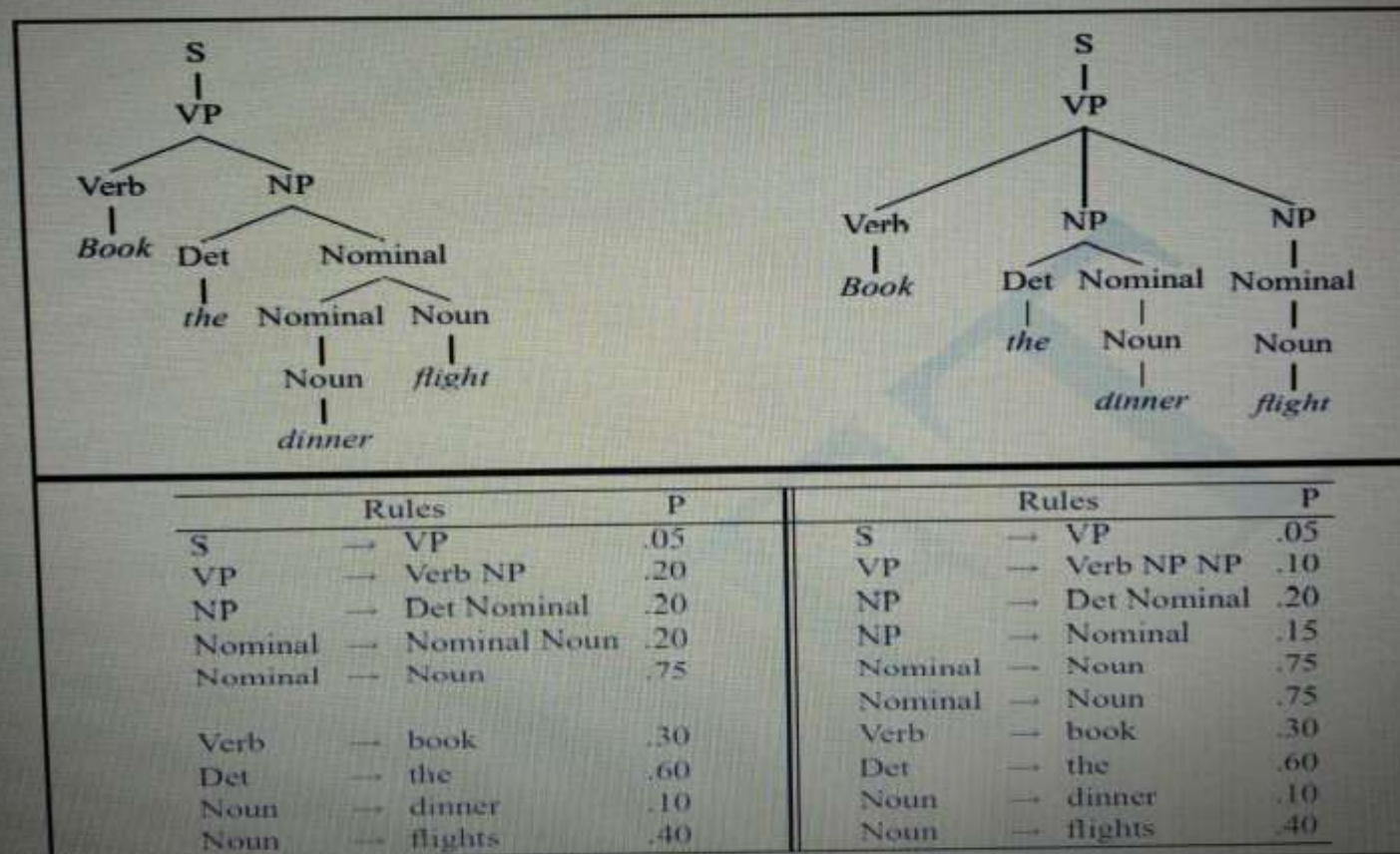
**Figure 14.1** A PCFG which is a probabilistic augmentation of the  $\mathcal{L}_1$  miniature English CFG grammar and lexicon of Fig. ?? in Ch. 13. These probabilities were made up for pedagogical purposes and are not based on a corpus (since any real corpus would have many more rules, and so the true probabilities of each rule would be much smaller).

## Disambiguation with PCFGs

- PCFG assigns probability to each parse tree (ie. Each derivation) of a sentence  $S$
- The probability of a particular parse  $T$  is defined as a product of the probabilities of all the  $n$  rules used to expand each of the non-terminal nodes in the parse tree  $T$ , where each rule  $i$  can be expressed as  $LHS_i = RHS_i$

$$P(T, S) = \prod_{i=1}^n P(RHS_i | LHS_i)$$

Refer figure for ex statement “Book the dinner flight”



**Figure 14.2** Two parse trees for an ambiguous sentence. The transitive parse (a) corresponds to the sensible meaning "Book flights that serve dinner", while the ditransitive parse (b) to the nonsensical meaning "Book flights on behalf of 'the dinner'"

# PoS Tagging using Hidden Markov Model (HMM)

1) An easy introduction to Hidden Markov Model (HMM) - Part 1

<https://www.youtube.com/watch?v=YIL0YARYK-o>

2) [https://www.youtube.com/watch?v=7ak1\\_zDUgEg](https://www.youtube.com/watch?v=7ak1_zDUgEg)

# **TreeBank: A data driven approach to Syntax**

What is the meaning of treebank?

treebank (plural treebanks) **A database of sentences which are annotated with syntactic information, often in the form of a tree.**

A treebank can be defined as a linguistically annotated corpus that includes some grammatical analysis beyond the part-of-speech level



What is treebank dataset?

English Web Treebank is a dataset containing 254,830 word-level tokens and 16,624 sentence-level tokens of webtext in 1174 files annotated for sentence- and word-level tokenization, part-of-speech, and syntactic structure

SUSANNE Corpus (Sampson 1995), where each token is represented by one line, with part-of-speech (including morphosyntactic features) in the first column, the actual token in the second column, and the lemma in the third column

|       |            |            |
|-------|------------|------------|
| AT    | The        | the        |
| JJ    | grand      | grand      |
| NN1c  | jury       | jury       |
| VVDv  | took       | take       |
| AT1   | a          | a          |
| NN1c  | swipe      | swipe      |
| II    | at         | at         |
| AT    | the        | the        |
| NNL1n | State      | state      |
| NN1u  | Welfare    | welfare    |
| NNJ1c | Department | department |
| GG    | +<apos>s   | -          |
| VVGt  | handling   | handle     |
| IO    | of         | of         |
| JJ    | federal    | federal    |
| NN2   | funds      | fund       |
| YG    | -          | -          |
| VVNt  | granted    | grant      |
| IF    | for        | for        |
| NN1c  | child      | child      |
| NN1u  | welfare    | welfare    |
| NN2   | services   | service    |
| II    | in         | in         |
| VV0t  | foster     | foster     |
| NN2   | homes      | home       |
| YF    | +,         | -          |

Fig. 13.1: Word-level annotation in the SUSANNE Corpus

# Treebank development

The methods and tools for treebank development have evolved considerably from the very first treebank projects, where all annotation was done manually, to the present-day situation, which is characterized by a more or less elaborate combination of manual work and automatic processing, supported by emerging standards and customized software tools

# Tools and Standards

Many of the software tools that are used in treebank development are tools that are needed in the development of any annotated corpus, such as tokenizers and part-of speech taggers.

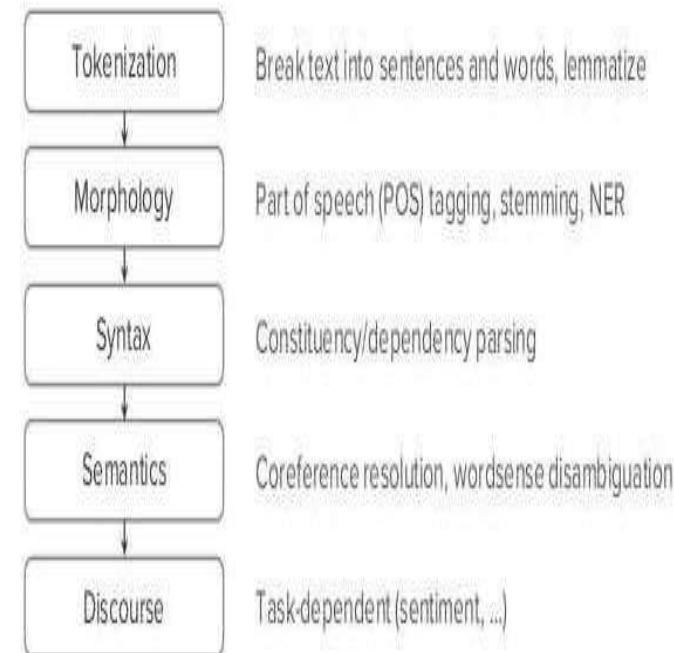
Tools that are specific to treebank development are primarily tools for syntactic preprocessing and specialized annotation tools.

- Well-known examples of **syntactic parsers** used in treebank development are the deterministic Fidditch parser (Hindle 1994), used in the development of the Penn Treebank, and the **statistical parser** of Collins et al. (1999), used for the Prague Dependency Treebank.
- It is also common to use **partial parsers (or chunkers)** for syntactic preprocessing, since partial parsing can be performed with higher accuracy than full parsing

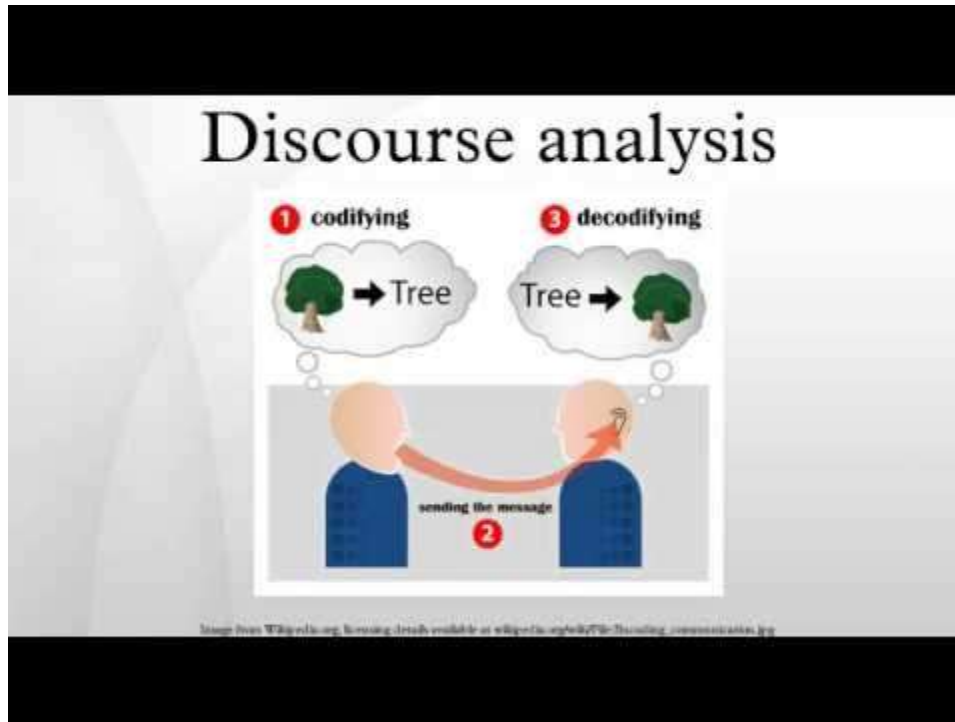
# 4. SEMANTIC ANALYSIS

- It refers to understanding what text means.
- Semantic analysis analyzes the grammatical format of sentences, including the arrangement of words, phrases, and clauses, to determine relationships between independent terms in a specific context.
- It is driving force behind machine learning tools like chatbots, search engines, and text analysis.

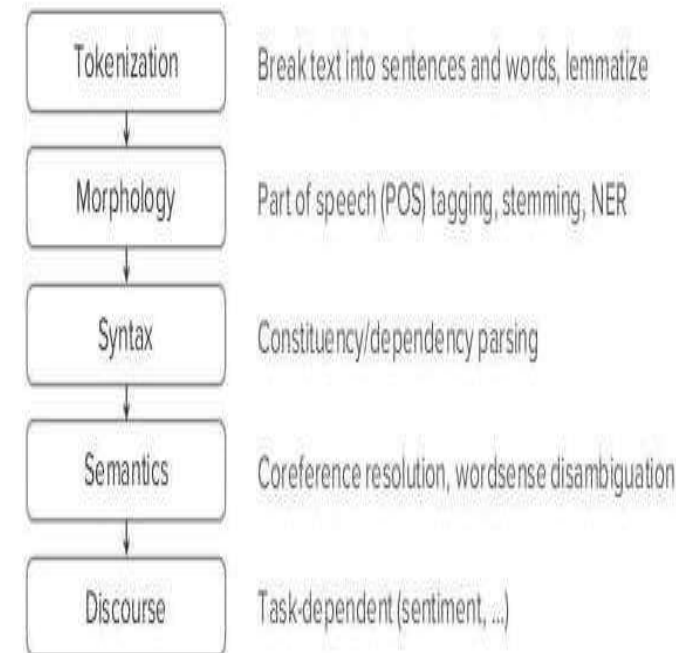
## "Classical" NLP Pipeline



# 5. DISCOURSE ANALYSIS



## “Classical” NLP Pipeline





# References

<https://www.ibm.com/in-en/topics/natural-language-processing>

**END OF UNIT 2**